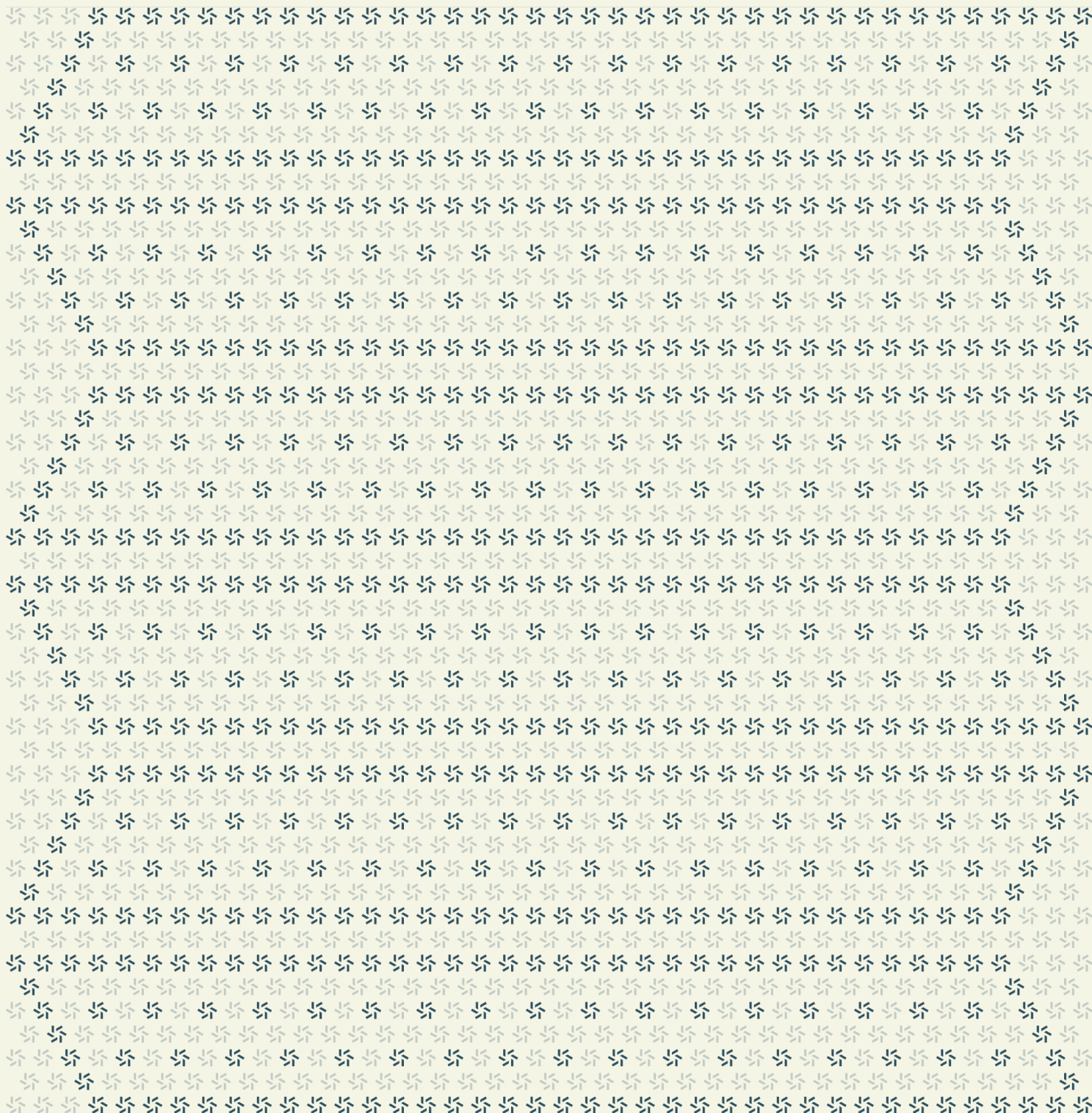


August 14, 2026

Multiliquid SVM

Solana Application Security Assessment



Contents

About Zellic	4
1. Overview	5
1.1. Executive Summary	5
1.2. Goals of the Assessment	5
1.3. Non-goals and Limitations	5
1.4. Results	5
2. Introduction	7
2.1. About Multiliquid SVM	7
2.2. Methodology	7
2.3. Scope	9
2.4. Project Overview	9
2.5. Project Timeline	10
3. Detailed Findings	11
3.1. Stale NAV can enable an atomic cross-asset Treasury arbitrage	11
3.2. Redemption during an APY posting gap can realize excess yield	14
3.3. The new GlobalConfig layout is incompatible with deployed Swap state	17
3.4. Token-2022 transfer fees cause the programs to account for tokens they never receive	19
3.5. Pruning referenced APY entries can permanently block position settlement	22
3.6. Public swaps can exhaust rent for a shared Treasury position and disable an LP's routes	24

3.7.	Fee asset pause status is not enforced during swaps	27
3.8.	Treasury deposit and redeem lack slippage and deadline protection	28
3.9.	close_pair performs an unguarded hook-aware zero-amount transfer	30
3.10.	Deposits can be accepted before the first APY entry makes redemption possible	31
3.11.	Failure of the fee-token NAV disables every swap	34
3.12.	Immediate LP fee changes can invalidate swaps without quote limits	36
3.13.	Treasury-token transfers do not transfer the corresponding redemption entitlement	38
3.14.	Treasury losses are realized by the users who redeem last	40
3.15.	NAV aggregation fails closed when any configured source is unavailable	43
3.16.	Global configuration initialization is not restricted to the upgrade authority	45
3.17.	Duplicate NAV sources can bias the aggregated price	47
3.18.	Closing one pair requires clearing the LP's shared Treasury position	49
3.19.	Manager's ambient signature is exposed to asset transfer hooks	51
4.	Threat Model	53
4.1.	Program: swap	53
4.2.	Program: treasury	95
5.	Assessment Results	127
5.1.	Disclaimer	127

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) [worldwide](#) in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io [and follow @zellic_io](#) [on Twitter](#). If you are interested in partnering with Zellic, contact us at hello@zellic.io [.](#)



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Uniform Labs from July 21st to August 7th, 2026. During this engagement, Zellic reviewed Multiliquid SVM's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Does the swap function use the correct price?
 - Does the swap function use the correct decimal conversion?
 - Does the swap function use the correct exchange-rate calculation?
 - Does the yield-calculation function calculate yield correctly?
-

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody

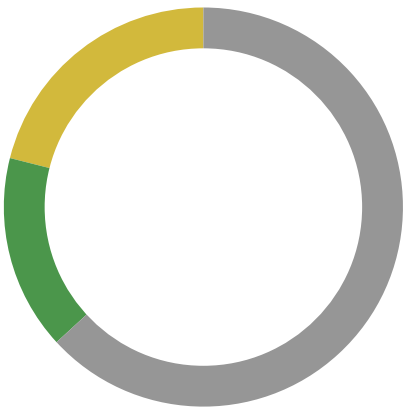
Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

1.4. Results

During our assessment on the scoped Multiliquid SVM programs, we discovered 19 findings. No critical issues were found. Four findings were of medium impact, three were of low impact, and the remaining findings were informational in nature.

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	0
Medium	4
Low	3
Informational	12



2. Introduction

2.1. About Multiliquid SVM

Uniform Labs contributed the following description of Multiliquid SVM:

Multiliquid operates as neutral market infrastructure, facilitating fixed-price swaps based on Net Asset Value between tokenized assets and stablecoins. The protocol sources liquidity directly from integrated stablecoin issuers and balance sheet providers, eliminating traditional DeFi inefficiencies.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the programs.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect

its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

2.3. Scope

The engagement involved a review of the following targets:

Multiliquid SVM Programs

Type	rust
Platform	SVM
Target	Multiliquid-SVM
Repository	https://github.com/uniformlabs/Multiliquid-SVM ↗
Version	54ab1a959ab42ba0dc04bc38ecdc776760de9a44
Programs	treasury swap

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 4.3 person-weeks. The assessment was conducted by two consultants over the course of three calendar weeks.

Contact Information

The following project manager was associated with the engagement:

Pedro Moura
✈ Engagement Manager
pedro@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Rajvardhan Agarwal
✈ Engineer
raj@zellic.io ↗

Xuehao Guo
✈ Engineer
guo@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

July 21, 2026	Kick-off call
----------------------	---------------

June 21, 2026	Start of primary review period
----------------------	--------------------------------

August 7, 2026	End of primary review period
-----------------------	------------------------------

3. Detailed Findings

3.1. Stale NAV can enable an atomic cross-asset Treasury arbitrage

Target	SVM Treasury program		
Category	Business Logic	Severity	Medium
Likelihood	Low	Impact	Medium

Description

The Treasury records deposits and settled positions as fungible USD-denominated credit rather than as claims against the asset originally deposited. In particular, a pending deposit stores its USD value, timestamp, and asset type, but not its mint.

```
// programs/treasury/src/states/user_position.rs#L4-L16
pub struct PendingDeposit {
    pub amount_deposited_fp: u128,
    pub deposit_ts: i64,
    pub asset_type: AssetType,
}
```

The deposit instruction obtains the input asset's NAV and uses the result to calculate both the USD credit and the number of Treasury tokens minted.

```
// programs/treasury/src/instructions/deposit.rs#L121-L184
let nav_9dec = self.asset_config.get_nav(nav_accounts)?;

let (asset_amount, deposit_value_fp, treasury_amount) = match exact_mode {
    ExactMode::ExactIn => {
        let deposit_value_fp =
            asset_amount_to_value_fp(amount, nav_9dec,
self.asset_mint.decimals)?;
        let treasury_amount =
            treasury_amount_from_value_fp(deposit_value_fp,
treasury_mint.decimals)?;

        (amount, deposit_value_fp, treasury_amount)
    }
    // ...
};
```

Redemption is not bound to that input asset. The caller selects any supported asset_config and

asset_mint, and the program converts the USD-denominated Treasury amount using the selected output asset's NAV. The instruction also consumes pending deposits before settled credit without checking whether their yield-start timestamp has elapsed.

```
// programs/treasury/src/instructions/redeem.rs#L104-L206
let nav_9dec = self.asset_config.get_nav(nav_accounts)?;
let (treasury_amount, redeem_asset_amount) = match exact_mode {
    ExactMode::ExactIn => {
        let treasury_amount = amount;
        let redeem_asset_amount = asset_amount_from_treasury_amount(
            treasury_amount,
            nav_9dec,
            self.asset_mint.decimals,
            treasury_mint_decimals,
        )?;

        (treasury_amount, redeem_asset_amount)
    }
    // ...
};

// ...

let (remaining_pending_deposits, consumed_pending) = consume_pending_deposits(
    remaining_pending_deposits,
    treasury_amount_to_value_fp(treasury_amount, treasury_mint_decimals)?,
    treasury_mint_decimals,
)?;
```

Consequently, if the NAV of supported asset A remains materially above its realizable value while supported asset B is priced accurately, an attacker can deposit A, receive overstated USD credit, and immediately redeem B. Both instructions can execute in the same Solana transaction because the second instruction observes the tokens and position state written by the first. The attacker can repeat this process while the inaccurate NAV remains accepted.

The protocol includes freshness checks for Pyth and Redstone sources, but U64FixedAddress has no timestamp. An excessively permissive freshness configuration, an incorrect oracle update, or a stale fixed-address source can therefore supply the required overvaluation. The attack does not require the attacker to control the oracle once such a state exists.

Impact

An attacker can exchange an overvalued asset for accurately priced assets at the Treasury's expense. Since positions are not isolated by mint, the attack can consume the available balances of healthy asset vaults and transfer losses to unrelated depositors. The amount extractable is bounded by the liquidity in those vaults and by the magnitude and duration of the NAV

discrepancy.

Recommendations

Require every NAV source to pass a strict on-chain freshness check, including an authenticated timestamp and maximum age for fixed-address prices. Bind each pending deposit to its source mint and prohibit cross-asset redemption until the deposit matures, preventing credit created from a stale input NAV from being atomically converted into healthy vault assets.

Remediation

This issue has been acknowledged by Uniform Labs, and they have provided the following response:

The protocol design assumes assets with changing NAVs use properly updated oracles (Pyth, Redstone) with appropriate freshness limits, while U64FixedAddress is intended for assets expected to maintain constant NAV, such as a money market fund structured to maintain a \$1 peg.

3.2. Redemption during an APY posting gap can realize excess yield

Target	SVM Treasury program		
Category	Business Logic	Severity	Medium
Likelihood	Medium	Impact	Medium

Description

When daily APY entries have not been posted through the current time, the redeem instruction extrapolates the latest posted APY from `last_yield_update_ts`. The extrapolation continues until the current timestamp or `max_unposted_yield_seconds`, whichever occurs first.

```
// programs/treasury/src/instructions/redeem.rs#L137-L158
let posted_index_fp = self.global_config.global_yield_index_fp;
let posted_until_ts = self.global_config.last_yield_update_ts;
let max_extrapolation_secs =
    i64::try_from(self.global_config.max_unposted_yield_seconds)
        .map_err(|_| TreasuryErrors::ArithmeticError)?;
let extrapolation_deadline_ts = posted_until_ts
    .checked_add(max_extrapolation_secs)
    .ok_or(TreasuryErrors::ArithmeticError)?;
let effective_ts = now_ts.min(extrapolation_deadline_ts);
let current_yield_index_fp = redeem_global_credit_index_fp(
    posted_index_fp,
    posted_until_ts,
    self.latest_apy_entry.apy_bps,
    effective_ts,
)?;
```

The resulting index settles the caller's entire position, even when the caller redeems only part of the position. The settlement index is written back to `global_yield_index_on_last_update`.

```
// programs/treasury/src/instructions/redeem.rs#L169-L193
let (
    settled_principal_amount,
    settled_accumulated_yield,
    current_index_basis,
    // ...
) = settle_user_position(
    // ...
    current_yield_index_fp,
```

```
        effective_ts,  
        Some(self.latest_apy_entry.apy_bps),  
        // ...  
    )?;  
  
    user_position.global_yield_index_on_last_update = current_index_basis;
```

The oracle may subsequently correct an earlier APY entry and recompute the official global index. However, position settlement explicitly takes the maximum of the official index and the user's saved index, so a correction cannot reduce yield that was previously recorded for the user.

```
// programs/treasury/src/math.rs#L238-L245  
let principal_settlement_index =  
    current_index_fp.max(user_index_on_last_update);
```

If the most recently posted APY is higher than the rate later assigned to the missing period, a user can redeem during the posting gap and receive the difference. Redemption burns principal Treasury tokens after subtracting consumed yield, which permits the extrapolated yield to be paid without requiring an equivalent amount of principal tokens. Once the asset transfer completes, a later APY correction cannot recover the excess amount.

The default extrapolation cap is seven days. At the maximum permitted 100% APY, seven days of extrapolation represents approximately 1.34% growth, although the manager can configure a different cap. The issue therefore exposes a bounded percentage of affected positions under the default parameters rather than creating arbitrary principal.

Impact

Users who redeem during an oracle posting gap can receive more yield than the subsequently corrected APY history supports. The overpayment is taken from Treasury liquidity and cannot be recovered on chain after redemption. Exploitation requires both a posting gap and a later downward difference in the applicable APY.

Recommendations

Exclude APY extrapolated across an unposted period from redeemable yield. Track that yield as provisional and release it only after the corresponding daily APY entries have been posted and finalized, so a later downward rate cannot leave an unrecoverable overpayment.

Remediation

Uniform Labs updated redemption accounting so that the extrapolated index used to price a redemption is not written to the remaining position. This prevents a small redemption from

crediting extrapolated yield to the entire remaining position. Uniform Labs confirmed that allowing the current redemption to realize extrapolated yield within `max_unposted_yield_seconds` is intended behavior.

This issue has been acknowledged by Uniform Labs, and a fix was implemented in commit [26df8636](#) [↗](#).

3.3. The new GlobalConfig layout is incompatible with deployed Swap state

Target	SVM Swap program		
Category	Code Maturity	Severity	Medium
Likelihood	Low	Impact	Medium

Description

The Swap program's persistent GlobalConfig layout was extended by inserting `fee_token_mint` and `spread_protocol_fees_bps` around the existing bump field rather than consuming compatible padding at the end of the account.

```
// programs/swap-program/src/state/global_config.rs#L7-L25
pub struct GlobalConfig {
    pub admin: Pubkey,
    pub fee_wallet: Pubkey,
    pub protocol_fees_bps: u16,
    pub paused: bool,
    pub pending_new_admin: Option<Pubkey>,
    pub fee_token_mint: Pubkey,
    pub bump: u8,
    pub spread_protocol_fees_bps: u16,
    pub padding: [u8; 128],
}
```

The deployed GlobalConfig account uses the previous layout and has a 237-byte allocation. When `pending_new_admin` is `None`, deserializing the new layout requires at least 239 bytes, while `8 + GlobalConfig::INIT_SPACE` for a newly created account is 271 bytes. Anchor therefore rejects the deployed account before an instruction handler can run.

Increasing the account by only two bytes would not resolve the incompatibility. In the previous layout, `bump` immediately followed `pending_new_admin`; under the new layout, that byte becomes the first byte of `fee_token_mint`, and every subsequent field is shifted. The account must be decoded using its previous schema and rewritten using the new schema.

The repository does not contain an instruction that performs this migration. The existing initialization instruction uses `init` for the fixed global PDA and can only create a new account, so it cannot repair the deployed one.

```
// programs/swap-program/src/instructions/init_global_config.rs#L10-L19
#[account(
```

```
init,  
payer = admin,  
space = 8 + GlobalConfig::INIT_SPACE,  
seeds = [GLOBAL_CONFIG_PREFIX],  
bump,  
)]  
pub global_config: Account<'info, GlobalConfig>,
```

If the reviewed binary is deployed without a preceding migration, almost every Swap instruction that accepts `Account<GlobalConfig>` fails during account validation. Since failure occurs before state is written, the upgrade authority can recover by redeploying a compatible binary, but the program remains unavailable until that action is completed.

Impact

Deploying the reviewed Swap binary over the existing program halts swaps and administrative operations that deserialize `GlobalConfig`. LPs are also unable to use the affected program instructions to withdraw liquidity while the incompatible binary is active. The issue requires an action by the trusted upgrade authority and does not corrupt the account before failure, which permits recovery through another upgrade.

Recommendations

Implement an explicit legacy-to-new `GlobalConfig` migration that deserializes the 237-byte legacy layout, reallocates the account to `8 + GlobalConfig::INIT_SPACE`, and writes every preserved and new field at its intended offset. Test the migration against a fixture built from the deployed 237-byte account data, and block deployment unless the fixture migrates and deserializes successfully.

Remediation

This issue has been acknowledged by Uniform Labs, and a fix was implemented in commit [2acad6fb](#).

3.4. Token-2022 transfer fees cause the programs to account for tokens they never receive

Target	SVM Treasury and Swap programs		
Category	Integration Risks	Severity	Medium
Likelihood	Low	Impact	Medium

Description

The Treasury and Swap programs accept both legacy SPL Token and Token-2022 mints through the token-interface abstraction. Neither asset-registration path rejects a mint that contains the Token-2022 TransferFeeConfig extension.

A transfer-fee mint debits the requested gross amount from its source but credits a smaller spendable amount to the destination. The Treasury nevertheless calculates USD credit from the requested amount before the transfer, then mints the corresponding gross amount of Treasury tokens without measuring the asset vault's actual balance increase.

```
// programs/treasury/src/instructions/deposit.rs#L165-L218
let (asset_amount, deposit_value_fp, treasury_amount) = match exact_mode {
    ExactMode::ExactIn => {
        let deposit_value_fp =
            asset_amount_to_value_fp(amount, nav_9dec,
self.asset_mint.decimals)?;
        let treasury_amount =
            treasury_amount_from_value_fp(deposit_value_fp,
treasury_mint_decimals)?;

        (amount, deposit_value_fp, treasury_amount)
    }
    // ...
};

transfer_asset_to_vault(
    &self.asset_token_program,
    &self.user_asset_token_account,
    &self.asset_vault,
    &self.asset_mint,
    &self.user.to_account_info(),
    asset_amount,
    transfer_hook_accounts,
)?;
```

```
mint_to(cpi_ctx, treasury_amount)?;
```

Depositing a fee-bearing asset therefore creates USD credit and Treasury tokens for the gross amount even though the vault receives only the net amount. Because Treasury credit can be redeemed for another supported asset, the shortfall can be transferred to an otherwise healthy vault. If the depositor also benefits from the transfer-fee authority, the withheld amount can be recovered outside the Treasury.

The Swap program has the same nominal-versus-net mismatch. It calculates `amount_in_for_vault` and `amount_out`, checks the nominal output against `min_amount_out`, and then submits transfers for those nominal amounts. The common transfer helper does not return or validate the recipient's actual balance delta.

```
// programs/swap-program/src/instructions/swap.rs#L464-L590
if let Some(min_amount_out) = min_amount_out {
    require!(amount_out >= min_amount_out, ErrorCode::AmountOutTooLow);
}

// ...

// from vault to user
spl_helpers::transfer(
    token_program_out,
    from_out,
    to_out,
    mint_out,
    &ctx.accounts.lp_vault_authority.to_account_info(),
    amount_out,
    Some(&signer),
    transfer_remaining_accounts,
)?;

// from user to vault
spl_helpers::transfer(
    token_program_in,
    from_in,
    to_in,
    mint_in,
    &ctx.accounts.user.to_account_info(),
    amount_in_for_vault,
    None,
    transfer_remaining_accounts,
)?;
```

A fee-bearing input causes the LP to provide output priced against more input than it receives. A

fee-bearing output causes the user to receive less than the amount against which the program enforced `min_amount_out`, and an exact-output swap no longer guarantees the requested net output.

The deployment records reviewed during the engagement did not list a mint with `TransferFeeConfig`. Exploitation therefore requires a trusted administrator or manager to list such a mint. Once listed, however, a transfer-fee authority may also change a zero fee to a nonzero fee without changing the mint address.

Impact

If a transfer-fee mint is listed, repeated deposits or swaps can create persistent deficits for Treasury vaults or LPs. Cross-asset redemption can propagate the deficit to other assets, while output-side fees invalidate user slippage expectations. The condition is not reachable with the assessed asset set, and an authorized configuration action is required before it can arise.

Recommendations

Reject mints containing the Token-2022 `TransferFeeConfig` extension on chain in both asset-registration paths unless complete transfer-fee accounting is deliberately implemented. Apply the check consistently to base, quote, protocol-fee, Treasury-asset, and Treasury-token mints so unsupported fee-bearing assets cannot enter either program.

Remediation

This issue has been acknowledged by Uniform Labs, and a fix was implemented in commit [d06015f6](#).

3.5. Pruning referenced APY entries can permanently block position settlement

Target	SVM Treasury program		
Category	Business Logic	Severity	Low
Likelihood	Low	Impact	Low

Description

The manager can close an APY entry after its retention period has elapsed, provided that the entry is not the latest entry. The instruction does not verify on chain whether a pending deposit still depends on the entry. A source-code comment acknowledges that the manager must perform this check off-chain.

```
// programs/treasury/src/instructions/prune_apy_entry.rs#L25-L49
/// SAFETY: Before calling, verify off-chain (via getProgramAccounts) that no
/// UserPosition has a pending deposit referencing this day – pruning it will
/// brick those users permanently.
pub fn prune_apy_entry(&mut self, _day_start_ts: u64) -> Result<()> {
    require!(
        self.apy_entry.key() != self.global_config.latest_apy_entry,
        TreasuryErrors::CannotPruneLatestApyEntry
    );

    // ...
    require!(
        current_timestamp >= retention_end_ts,
        TreasuryErrors::ApyEntryRetentionNotMet
    );
}
```

When a stable pending deposit becomes eligible for settlement, the program reconstructs its starting index from the APY entry for the corresponding day. A missing entry causes the entire settlement to fail.

```
// programs/treasury/src/math.rs#L133-L137
if earning_start_day_start_ts < posted_until_ts {
    let apy_entry = find_apy_entry_for_day(apy_entries,
        earning_start_day_start_ts)
        .ok_or(TreasuryErrors::MissingApyEntry)?;

    stable_basis_index_fp(apy_entry, earning_start_ts,
```

```
    earning_start_day_start_ts)?  
}
```

The same settlement logic is reached through `deposit`, `redeem`, and the permissionless `accrue_interest` instruction. Calling `accrue_interest` before pruning can merge the record and prevent the failure, but it cannot repair a position after the required account has been closed. The manager's `admin_update_user_yield` instruction changes only accumulated yield and the saved global index; it does not remove or reconstruct pending deposits.

The configured retention period reduces the probability of the issue, and only the trusted manager can prune an entry. Nevertheless, the program accepts a retention duration only slightly above seven days, and correctness ultimately depends on a complete off-chain enumeration of all variable-length position accounts before every prune.

Impact

An operational omission during pruning can permanently prevent affected users from settling their positions through the normal instruction set. Since settlement is part of redemption, both principal and yield can remain inaccessible until the program is upgraded or the account state is migrated. The failure can also reach Treasury-backed Swap routes that use the affected LP position.

Recommendations

Maintain an on-chain reference count for each APY day, incrementing it when a pending deposit is created and decrementing it when the deposit is merged or consumed. Require the count to be zero before `prune_apy_entry` can close the corresponding APY account.

Remediation

This issue has been acknowledged by Uniform Labs, and they have provided the following response:

All pending deposits are expected to be merged before any pruning happens, user history is expected to be updated, and pruning is only expected to happen on multi-year-old rates.

3.6. Public swaps can exhaust rent for a shared Treasury position and disable an LP's routes

Target	SVM Treasury and Swap programs		
Category	Coding Mistakes	Severity	Low
Likelihood	Low	Impact	Low

Description

After an LP enables Treasury routing, every eligible stable-input swap deposits the received stable asset into the Treasury. The Swap program passes the transaction's trader as the funding payer but passes the LP vault-authority PDA as the Treasury user.

```
// programs/swap-program/src/instructions/swap.rs#L592-L613
treasury_deposit_input(
    treasury_accounts,
    ctx.accounts.user.to_account_info(),
    ctx.accounts.lp_vault_authority.to_account_info(),
    mint_in.to_account_info(),
    to_in.to_account_info(),
    token_program_in.to_account_info(),
    &signer,
    amount_in_for_vault,
    // ...
)?;
```

The LP vault authority is derived only from the liquidity-provider key, and the Treasury UserPosition is derived only from its user. Neither derivation includes a pair or asset mint, so the pending-deposit queue is shared across the LP's Treasury-backed routes.

A new position contains 50 pending-deposit slots and grows by another 50 slots when full. Before appending a record, deposit attempts to settle mergeable records. If the queue remains full, it reallocates the account and charges the incremental rent to the Treasury user, which is the LP PDA in this integration.

```
// programs/treasury/src/instructions/deposit.rs#L220-L241
if needs_grow {
    grow_user_position(
        &self.user_position.to_account_info(),
        &self.user.to_account_info(),
        &self.system_program.to_account_info(),
```

```
    )?;

    let mut user_position = self.user_position.load_mut()?;
    user_position.max_pending_deposits = user_position
        .max_pending_deposits
        .checked_add(PENDING_DEPOSITS_CHUNK)
        .ok_or(TreasuryErrors::PendingDepositCountTooLarge)?;
}
```

The Swap funding helper calculates the required rent only while the position or Treasury-token account is empty. After both accounts exist, it does not reserve lamports for later reallocations.

```
// programs/swap-program/src/treasury_cpi.rs#L108-L125
if treasury_user_position.data_is_empty() {
    required_lamports = required_lamports
        .checked_add(rent.minimum_balance(8 + treasury::UserPosition::LEN))
        .ok_or(ErrorCode::MathOverflow)?;
}

if user_treasury_token_account.data_is_empty() {
    required_lamports = required_lamports
        .checked_add(rent.minimum_balance(treasury_token_account_len(&treasury_mint)?))
        .ok_or(ErrorCode::MathOverflow)?;
}

let current_lamports = lp_vault_authority.lamports();
if current_lamports >= required_lamports {
    return Ok(());
}
```

An unprivileged trader can submit ordinary stable-input swaps while earlier deposits remain pending, filling the available slots one by one. When the next swap reaches a growth boundary, the Treasury CPI fails if the LP PDA lacks the additional rent. The transaction reverts and leaves the queue full, so later Treasury-backed stable-input swaps encounter the same failed reallocation.

Impact

A trader can disable the affected LP's Treasury-backed stable-input routes until the LP funds its PDA, disables Treasury routing, consumes pending credit, or waits until records can be settled. Since the position is shared, activity through one pair can disrupt other eligible routes belonging to the same LP. Other LPs and routes that do not invoke a Treasury deposit remain available, and the condition is recoverable but repeatable.

Recommendations

Separate the Treasury position owner from the rent payer: retain the LP PDA as the owner, but charge initialization and reallocation rent to a distinct signer. The Swap integration should use the initiating trader as that payer, and tests should cover an unfunded LP PDA crossing a reallocation boundary.

Remediation

This issue has been acknowledged by Uniform Labs, and fixes were implemented in the following commits:

- [ce412544](#) ↗
- [3e16735d](#) ↗
- [b11b7570](#) ↗

3.7. Fee asset pause status is not enforced during swaps

Target	swap-program		
Category	Coding Mistakes	Severity	Low
Likelihood	Low	Impact	Low

Description

swap enforces the pause flag for the global config, the pair, and the base and quote assets (plus their LpStableConfig), but never checks `fee_config.paused`. It nonetheless reads `fee_config.get_nav(...)` and uses `fee_nav` to convert the protocol fee into fee-token units and transfer it from the LP fee reserve to the protocol fee account. When the globally configured fee mint is distinct from both pair assets, its `AssetConfig` is only referenced as `fee_config`, so its pause flag is never consulted (if the fee mint equals the base or quote mint, the existing checks cover it). An admin pausing the fee asset therefore does not stop swaps from valuing and moving it.

Impact

While the fee asset is paused, swaps keep converting protocol fees at its NAV. A wrong-but-nonzero fee NAV over- or under-values the fee, over-deducting the LP's fee reserve or under-collecting protocol fees. The effect is bounded to per-swap fee amounts (bps) and capped by the LP fee account balance, with no impact on principal or vault reserves; `get_nav`'s divergence check still reverts on a zero or divergent NAV, so only mispricings that pass those checks apply. The practical concern is that the admin's pause lever is silently ineffective for the fee-valuation path.

Recommendations

Check `fee_config.paused` (and the corresponding `LpStableConfig.paused` if the fee token is Stable) before reading the fee NAV and transferring protocol fees.

Remediation

This issue has been acknowledged by Uniform Labs, and a fix was implemented in commit [28b279b3](#).

3.8. Treasury deposit and redeem lack slippage and deadline protection

Target	treasury		
Category	Code Maturity	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The public deposit and redeem instructions accept only amount and exact_mode; they have no minimum-output, maximum-input, or deadline parameter. The counter amount (Treasury tokens minted, or underlying assets returned) is computed from the asset's live NAV at execution time. If the NAV changes between the user quoting/signing and the transaction being included, an oracle update, or a manager set_asset_config NAV change, which has no timelock, the user silently executes at the new rate: fewer Treasury tokens or assets received, or more paid on an exact-output conversion. NAV is oracle-sourced and cannot be moved by a third party within the transaction, so there is no external sandbox/MEV vector; the exposure is honest NAV drift and manager repricing. The sibling swap instruction already provides min_amount_out/max_amount_in, so the protection pattern exists but is absent here.

Impact

Users can receive a worse conversion than quoted, bounded by the NAV movement over the inclusion window. For the intended slow-moving (e.g. MMF) NAVs this is small; it grows with more volatile assets or a manager NAV change. No principal beyond the rate delta is at risk.

Recommendations

Add optional min_amount_out (and max_amount_in for exact-output modes) enforced after the amounts are computed, mirroring swap.

Remediation

This issue has been acknowledged by Uniform Labs, and fixes were implemented in the following commits:

- [9bbfd445](#) ↗
- [7df9cd15](#) ↗
- [f9b36cd6](#) ↗
- [487b0db0](#) ↗

- [4361c1f6 ↗](#)

3.9. close_pair performs an unguarded hook-aware zero-amount transfer

Target	swap-program		
Category	Coding Mistakes	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

When closing a pair, `close_pair` returns each vault's balance to the LP via `spl_helpers::transfer` using `*_vault_token_account.amount` as the amount, with no check that the amount is non-zero. Since the LP must call `remove_liquidity` first, the vault balance at teardown is typically zero, so this is a zero-amount transfer. `spl_helpers::transfer` always routes through `onchain::invoke_transfer_checked`, and Token-2022 still invokes the mint's configured transfer hook for a zero-amount transfer. A hook that rejects zero-value transfers, or whose program/extra accounts are unavailable during teardown, makes the transfer revert, which fails the whole instruction and leaves the (otherwise empty) pair, its vault token accounts, and the `UserVaultInfo` PDAs unclosable.

This only affects pairs whose base/quote mint uses a transfer hook; standard and no-hook mints are unaffected.

Impact

The LP cannot close an empty pair and cannot reclaim the associated rent (vault token accounts, pair, and vault-info PDAs). It is a rent/liveness issue on the LP's own accounts, no principal, vault, or protocol funds are at risk. However, if the hook permanently rejects zero-amount transfers, the rent is stuck indefinitely.

Recommendations

Skip the transfer when the vault balance is zero (guard each `spl_helpers::transfer` with `if amount > 0`), and proceed directly to closing the token account.

Remediation

This issue has been acknowledged by Uniform Labs, and a fix was implemented in commit [04243621](#).

3.10. Deposits can be accepted before the first APY entry makes redemption possible

Target	SVM Treasury program		
Category	Code Maturity	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

Treasury initialization stores the default public key in `latest_apy_entry` and makes the global configuration active immediately. Asset initialization likewise creates each supported asset in an unpaused state.

```
// programs/treasury/src/instructions/initialize.rs#L91-L106
self.global_config.set_inner(GlobalConfig {
    // ...
    global_yield_index_fp: GLOBAL_CREDIT_INDEX_ONE,
    last_yield_update_ts: get_day_start_ts(clock.unix_timestamp)?,
    latest_apy_entry: Pubkey::default(),
    // ...
    paused: false,
    bump: bumps.global_config,
});

// programs/treasury/src/instructions/initialize_asset_config.rs#L72-L79
self.asset_config.set_inner(AssetConfig {
    asset_mint: self.asset_mint.key(),
    asset_type,
    paused: false,
    nav_data,
    price_difference_bps,
    bump: bumps.asset_config,
});
```

The `deposit` instruction requires neither a latest APY account nor a nondefault `latest_apy_entry`, so deposits can succeed after the trusted manager registers an asset. In contrast, Anchor validates a concrete `Account<ApyEntry>` for every redemption and requires its key to equal the value stored in the global configuration.

```
// programs/treasury/src/instructions/redeem.rs#L80-L84
#[account(
```

```
constraint = latest_apy_entry.key() == global_config.latest_apy_entry
    @ TreasuryErrors::InvalidLatestApyEntry
)]
pub latest_apy_entry: Account<'info, ApyEntry>,
```

Before the first rate is posted, no account can satisfy both requirements. `Pubkey::default()` is the System Program address rather than a Treasury-owned APY entry, while any genuine `ApyEntry` has a different key. A user can therefore deposit but cannot redeem through the normal instruction.

The oracle authority cannot close this gap immediately. The first APY entry corresponds to the UTC day in which initialization occurred, and the posting instruction rejects it until that day has fully elapsed.

```
// programs/treasury/src/instructions/post_daily_rate.rs#L60-L81
if created {
    require!(
        day_start_ts_i64 == self.global_config.last_yield_update_ts,
        TreasuryErrors::OutOfOrderApyEntry
    );
    let clock = Clock::get()?;
    require!(
        clock.unix_timestamp
            >= day_start_ts_i64
                .checked_add(SECONDS_PER_DAY)
                .ok_or(TreasuryErrors::ArithmeticError)?,
        TreasuryErrors::DayNotElapsed
    );

    // ...
    self.global_config.latest_apy_entry = self.apy_entry.key();
}
```

The condition depends on the trusted deployment roles exposing an asset before completing the first rate post. Under the intended ordering, users need not encounter it.

Impact

Depositors can be temporarily unable to redeem during Treasury bootstrap. Under normal operation, the window lasts until the initialization day has elapsed and the oracle authority posts the first rate; an operational delay can extend it. Existing assets and deployments that already have an APY entry are unaffected.

Recommendations

Keep the Treasury and all asset configurations paused until `latest_apy_entry` references a valid `ApyEntry`, and reject deposits, global unpausing, and asset activation while the field remains the default key. Enforce this ordering on chain and add a deployment test confirming that redemption works before any user-facing asset accepts deposits.

Remediation

This issue has been acknowledged by Uniform Labs, and a fix was implemented in commit [e6ee4180](#).

3.11. Failure of the fee-token NAV disables every swap

Target	SVM Swap program		
Category	Business Logic	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

Every invocation of swap loads and validates the NAVs of the base token, quote token, and global fee token before calculating the swap's pair fee or protocol-fee amount.

```
// programs/swap-program/src/instructions/swap.rs#L366-L380
let base_nav = ctx.accounts.base_config.get_nav(cpi_remaining_accounts)?;
if base_nav == 0 {
    return Err(ErrorCode::InvalidNav.into());
}

let quote_nav = ctx.accounts.quote_config.get_nav(cpi_remaining_accounts)?;
if quote_nav == 0 {
    return Err(ErrorCode::InvalidNav.into());
}

let fee_nav = ctx.accounts.fee_config.get_nav(cpi_remaining_accounts)?;
if fee_nav == 0 {
    return Err(ErrorCode::InvalidNav.into());
}
```

The program does this even when both `protocol_fees_bps` and `spread_protocol_fees_bps` are zero, or when the calculated protocol fee would otherwise be zero. The conditional transfer of the fee token occurs only later.

```
// programs/swap-program/src/instructions/swap.rs#L434-L543
let protocol_fees = issuer_protocol_fees
    .checked_add(spread_protocol_fees)
    .ok_or(ErrorCode::MathOverflow)?;

// ...

if protocol_fees > 0 {
    require!(
        ctx.accounts.lp_fee_token_account.amount >= protocol_fees,
```

```
        ErrorCode::InsufficientLpFeeBalance
    );

    spl_helpers::transfer(
        &ctx.accounts.token_program_fee,
        &ctx.accounts.lp_fee_token_account,
        &ctx.accounts.fee_token_account,
        &ctx.accounts.fee_token_mint_address,
        &ctx.accounts.lp_fee_vault_authority.to_account_info(),
        protocol_fees,
        Some(&[fee_signer_seeds]),
        transfer_remaining_accounts,
    )?;
}
```

The fee token and its NAV configuration are global. If any configured fee-token source is unavailable, stale, malformed, or divergent enough for `get_nav` to fail, all pairs fail before reaching their own pricing and transfer logic. Setting protocol fees to zero does not provide a fallback because the unconditional NAV read remains in the execution path.

The failure requires a fee-token oracle or configuration incident, or an authorized update that introduces one. It is not caused directly by an arbitrary swap caller. The administrator can recover by correcting the NAV configuration or deploying a program change.

Impact

A failure in a single auxiliary fee-token NAV can halt swaps across every pair, including swaps for which no fee token would be transferred. LP withdrawals and instructions that do not execute the swap path are unaffected, and the outage is recoverable after the global dependency is restored.

Recommendations

Read and validate the fee-token NAV only after the swap calculates a nonzero fee amount. If both fee rates are zero or the fee rounds to zero, bypass the fee-token accounts and NAV; otherwise, require the NAV before collecting the fee. Add tests for unavailable fee-token NAV with zero and nonzero calculated fees.

Remediation

This issue has been acknowledged by Uniform Labs.

3.12. Immediate LP fee changes can invalidate swaps without quote limits

Target	SVM Swap program		
Category	Business Logic	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

This behaviour is documented by Uniform Labs as intended behaviour.

Each liquidity provider can change a pair's `redemption_fee_bps` and `discount_rate_bps` immediately. The update has no delay or pending phase and permits either field to reach 9,900 basis points, subject to their sum remaining at most 10,000 basis points.

```
// programs/swap-program/src/instructions/update_pair.rs#L37-L62
require!(redemption_fee_bps <= 9900, ErrorCode::OutOfRange);
require!(discount_rate_bps <= 9900, ErrorCode::OutOfRange);
require!(
    (redemption_fee_bps as u32) + (discount_rate_bps as u32) <= 10_000,
    ErrorCode::OutOfRange
);

let pair = &mut ctx.accounts.pair;
pair.redemption_fee_bps = redemption_fee_bps;
pair.discount_rate_bps = discount_rate_bps;
pair.paused = paused;
```

The swap interface supports `min_amount_out` and `max_amount_in`, but both protections are optional. When omitted, the instruction accepts whatever amounts result from the pair parameters at execution time.

```
// programs/swap-program/src/instructions/swap.rs#L464-L472
if let Some(min_amount_out) = min_amount_out {
    require!(amount_out >= min_amount_out, ErrorCode::AmountOutTooLow);
}
if let Some(max_amount_in) = max_amount_in {
    require!(
        amount_in_for_vault <= max_amount_in,
        ErrorCode::AmountInTooHigh
    );
}
```

```
    );  
}
```

A user may construct or sign a transaction based on one fee configuration, while the LP changes that configuration before the transaction executes. If the user omitted the relevant bound, the swap can execute on substantially worse terms. The LP controls the configuration but does not have deterministic transaction ordering, so exploitation depends on transaction timing and on a caller explicitly declining the available quote protection.

Impact

An affected user can receive substantially less output or pay substantially more input than expected. The loss is limited to swaps that omit the applicable bound, and robust clients can prevent it by always supplying a fresh `min_amount_out` or `max_amount_in`.

Recommendations

Make `min_amount_out` mandatory for exact-input swaps and `max_amount_in` mandatory for exact-output swaps. Reject transactions that omit the applicable bound, and have clients derive it from a fresh quote with an expiry so later LP fee changes cannot execute an unbounded swap.

Remediation

This issue has been acknowledged by Uniform Labs.

3.13. Treasury-token transfers do not transfer the corresponding redemption entitlement

Target	SVM Treasury program		
Category	Integration Risks	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The Treasury token is transferable through Token-2022, but the accounting state that authorizes redemption is stored separately in a UserPosition PDA derived from the owner's public key. A pending deposit records USD value, timing, and asset type within that owner-specific position.

```
// programs/treasury/src/states/user_position.rs#L4-L16
pub struct PendingDeposit {
    pub amount_deposited_fp: u128,
    pub deposit_ts: i64,
    pub asset_type: AssetType,
}

// programs/treasury/src/instructions/redeem.rs#L28-L34
#[account(
    mut,
    seeds = [USER_POSITION_SEED.as_bytes(), user.key().as_ref()],
    bump,
    constraint = user_position.load()?.owner == user.key()
)]
pub user_position: AccountLoader<'info, UserPosition>
```

Redemption requires both a Treasury-token account controlled by the signer and sufficient pending, principal, or yield credit in that same signer's position.

```
// programs/treasury/src/instructions/redeem.rs#L50-L56
#[account(
    mut,
    token::mint = treasury_mint,
    token::authority = user,
    token::token_program = treasury_token_program
)]
pub user_treasury_token_account: InterfaceAccount<'info, TokenAccount>
```

```
// ...  
  
require!(  
    remaining_redeem_amount <= settled_total,  
    TreasuryErrors::InsufficientRedeemableBalance  
);
```

Transferring Treasury tokens to another address does not move any corresponding `UserPosition` credit. The recipient therefore cannot redeem solely by presenting the transferred tokens, while the sender retains the position but may no longer hold enough tokens to burn the principal portion of a redemption. The parties can restore consistency by returning or otherwise reacquiring the tokens, but the transfer itself does not behave like a transfer of a bearer claim.

This separation is consistent with an account-bound credit design, and no protocol requirement establishes that the token must independently represent redemption entitlement. The risk arises primarily when wallets, integrations, or secondary markets infer ordinary bearer-token semantics from transferability.

Impact

A recipient or integrator that treats the Treasury token as an independently redeemable asset can acquire tokens that do not carry the expected claim. The original position owner can also make their own position temporarily unredeemable by transferring away the tokens required for the principal burn. The behavior does not create or steal value within the program by itself.

Recommendations

Prominently document in the protocol documentation, token metadata, UI, and SDK that the Treasury token is not a bearer claim: transferring it does not move the owner-bound `UserPosition` redemption entitlement. Wallet and integration surfaces should warn recipients and must not present the token as an independently redeemable asset.

Remediation

This issue has been acknowledged by Uniform Labs.

3.14. Treasury losses are realized by the users who redeem last

Target	SVM Treasury program		
Category	Protocol Risks	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The Treasury records user positions as fixed USD-denominated principal and yield. It does not mark those liabilities down when a supported asset loses value, a vault becomes undercollateralized, or the manager withdraws assets. The manager withdrawal path checks only the selected vault's token balance and does not enforce a reserve ratio against outstanding Treasury credit.

```
// programs/treasury/src/instructions/manager_withdraw.rs#L59-L86
require!(!self.global_config.paused, TreasuryErrors::ProgramPaused);
require!(amount > 0, TreasuryErrors::AmountMustBePositive);
require!(
    self.asset_vault.amount >= amount,
    TreasuryErrors::InsufficientVaultBalance
);

anchor_spl::token_interface::spl_token_2022::onchain::invoke_transfer_checked(
    &self.token_program.key(),
    self.asset_vault.to_account_info(),
    self.asset_mint.to_account_info(),
    self.manager_asset_token_account.to_account_info(),
    self.vault_authority.to_account_info(),
    remaining_accounts,
    amount,
    self.asset_mint.decimals,
    &signer,
)?;
```

A redeemer selects a supported output asset, and the program converts their USD claim using that asset's NAV. The transfer succeeds while the selected vault has enough tokens and fails once its balance is insufficient.

```
// programs/treasury/src/instructions/redeem.rs#L104-L291
let nav_9dec = self.asset_config.get_nav(nav_accounts)?;
let redeem_asset_amount = asset_amount_from_treasury_amount(
```

```
        treasury_amount,  
        nav_9dec,  
        self.asset_mint.decimals,  
        treasury_mint_decimals,  
    )?;  
  
    // ...  
  
    anchor_spl::token_interface::spl_token_2022::onchain::invoke_transfer_checked(  
        &self.asset_token_program.key(),  
        self.asset_vault.to_account_info(),  
        self.asset_mint.to_account_info(),  
        self.user_asset_token_account.to_account_info(),  
        self.vault_authority.to_account_info(),  
        transfer_hook_accounts,  
        redeem_asset_amount,  
        self.asset_mint.decimals,  
        &signer,  
    )?;
```

After an economic loss, early redeemers can therefore obtain full value while sufficient healthy liquidity remains, and later redeemers bear the resulting deficit. There is no on-chain pro rata loss allocation, reserve accounting, withdrawal queue, or insolvency mode that applies the impairment uniformly across outstanding claims.

The engagement materials describe the Treasury as a managed credit product and treat issuer solvency, asset quality, and manager withdrawals as trusted economic assumptions. Under that model, first-come loss allocation is a protocol-design consequence rather than a deviation from specified behavior.

Impact

If the Treasury becomes undercollateralized, users who redeem later may be unable to recover the full value recorded in their positions even though earlier users redeemed at par. The result can create a run incentive during an asset or issuer incident, but it does not arise from an unprivileged implementation exploit under the accepted trust model.

Recommendations

Disclose the accepted credit-risk and loss-ordering model, including that recorded USD claims are not automatically impaired and later redeemers may bear a deficit. Continuously monitor outstanding liabilities against realizable vault value, define explicit solvency and pause thresholds, and pause redemptions or manager withdrawals before reserves breach those thresholds.

Remediation

This issue has been acknowledged by Uniform Labs.

3.15. NAV aggregation fails closed when any configured source is unavailable

Target	SVM NAV data library		
Category	Protocol Risks	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

An asset may be configured with as many as five NAV sources. The shared `get_nav` implementation evaluates every configured entry sequentially, and any error immediately aborts the price calculation.

```
// crates/navdata/src/lib.rs#L265-L275
pub fn get_nav<'info>(<
    nav_data: &Vec<NavData>,
    remaining_accounts: &[AccountInfo<'info>],
    price_difference_bps: u16,
) -> Result<u64> {
    let mut nav_sum: u128 = 0;
    let mut max_nav: u128 = 0;
    let mut min_nav: u128 = u128::MAX;

    for nav_data in nav_data.iter() {
        let nav = match nav_data {
            // ...
        };
        // ...
    }
}
```

For example, a missing or stale Pyth account returns an error from inside the loop. Redstone deserialization, feed-identity, timestamp, and value failures behave similarly. The minimum, maximum, divergence check, and arithmetic mean are calculated only after every entry has succeeded.

```
// crates/navdata/src/lib.rs#L406-L430
let nav_u128 = nav as u128;
max_nav = max_nav.max(nav_u128);
min_nav = min_nav.min(nav_u128);
nav_sum = nav_sum
    .checked_add(nav_u128)
    .ok_or(ErrorCode::MathOverflow)?;
```

```
// after all sources have succeeded
if min_nav < min_acceptable_nav {
    return Ok(0);
}

let avg = nav_sum
    .checked_div(nav_data.len() as u128)
    .ok_or(ErrorCode::MathOverflow)?;
```

Configured sources therefore act as mandatory corroborators rather than availability fallbacks. If one source becomes stale, unavailable, or malformed, the affected asset cannot obtain a NAV even when every other source remains healthy. The shared library can consequently block Swap operations involving the asset and Treasury deposits or redemptions using it.

This behavior is not inherently incorrect. The reviewed implementation and project documentation do not establish that multiple sources are intended to provide quorum-based availability. Requiring all sources to agree within a divergence bound is a defensible fail-closed integrity policy; silently discarding failed sources could allow one remaining source to determine the price.

Impact

A natural failure in one configured oracle can make an asset temporarily unavailable even though another source continues publishing a valid price. Recovery requires the trusted administrator or manager to repair or replace the failed source. There is no permissionless price-manipulation or loss path inherent in the policy, and a caller that omits a required account only causes their own transaction to fail.

Recommendations

Document multiple NAV entries as mandatory all-of-N corroborators, and treat any failed source as an asset-level outage. Monitor every source and define a procedure for promptly pausing or updating an affected asset without weakening freshness, feed-identity, confidence, or divergence checks.

Remediation

This issue has been acknowledged by Uniform Labs.

3.16. Global configuration initialization is not restricted to the upgrade authority

Target	SVM Swap program		
Category	Code Maturity	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The Swap program's global initializer accepts any signer as admin. It does not receive the program's ProgramData account or verify that the signer is the program upgrade authority.

```
// programs/swap-program/src/instructions/init_global_config.rs#L10-L23
#[account(
    init,
    payer = admin,
    space = 8 + GlobalConfig::INIT_SPACE,
    seeds = [GLOBAL_CONFIG_PREFIX],
    bump,
)]
pub global_config: Account<'info, GlobalConfig>,

#[account(mut)]
pub admin: Signer<'info>,
```

The handler directly records that signer as the protocol administrator while accepting caller-selected fee settings.

```
// programs/swap-program/src/instructions/init_global_config.rs#L54-L77
let global_config = &mut ctx.accounts.global_config;

global_config.admin = ctx.accounts.admin.key();
global_config.fee_wallet = fee_wallet;
global_config.protocol_fees_bps = protocol_fees_bps;
global_config.fee_token_mint = fee_token_mint;
global_config.paused = true;
global_config.bump = ctx.bumps.global_config;
global_config.spread_protocol_fees_bps = spread_protocol_fees_bps;
```

On a fresh program ID, any party can submit the first initialization transaction for the deterministic global-config PDA. A successful first caller becomes the administrator, selects the fee wallet and

fee token, and prevents the intended initializer from using the same `init` instruction.

The Treasury initializer implements a stronger pattern by deriving the `ProgramData` account through the upgradable loader and requiring its upgrade authority to equal the initializing signer.

```
// programs/treasury/src/instructions/initialize.rs#L21-L33
#[account(
    seeds = [crate::ID.as_ref()],
    seeds::program =
    anchor_lang::solana_program::bpf_loader_upgradeable::id(),
    bump,
    constraint = program_data.upgrade_authority_address == Some(caller.key())
    @ TreasuryErrors::UnauthorizedInitializer,
)]
pub program_data: Account<'info, ProgramData>,
```

For the assessed Swap program ID, the global-config PDA had already been initialized before the review. Since the constraint uses `init`, an unprivileged caller cannot overwrite that account or reopen the first-caller window. The issue therefore applies to fresh deployments or deployments under a new program ID rather than the assessed instance.

Impact

An attacker who wins the initialization race on a fresh deployment can seize the program's configuration authority and exclude the intended administrator. The assessed instance is not exposed to that race, so this is recorded as deployment hardening rather than an active takeover vulnerability.

Recommendations

Require the upgradable-loader `ProgramData` account in `init_global_config` and verify that its upgrade authority matches the initializing signer, mirroring the Treasury initializer. The deployment process should also verify the resulting administrator and fee configuration before the program is funded.

Remediation

This issue has been acknowledged by Uniform Labs, and a fix was implemented in commit [308b1768](#).

3.17. Duplicate NAV sources can bias the aggregated price

Target	SVM NAV data library		
Category	Code Maturity	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The shared NAV validation function requires at least one source, caps the list at five, and validates the format and referenced account of each entry. It does not verify that the entries represent distinct accounts or feeds.

```
// crates/navdata/src/lib.rs#L156-L167
pub fn validate_nav<'info>(<
    nav_data: &Vec<NavData>,
    remaining_accounts: &[AccountInfo<'info>],
) -> Result<()> {
    if nav_data.is_empty() {
        return Err(ErrorCode::MustProvideAtLeastOneNavData.into());
    }
    if nav_data.len() > 5 {
        return Err(ErrorCode::Max5NavData.into());
    }

    for nav_data in nav_data {
        // validate each entry independently
        // ...
    }
}
```

At runtime, every vector entry contributes equally to `nav_sum`, and the result is divided by the vector length.

```
// crates/navdata/src/lib.rs#L406-L430
let nav_u128 = nav as u128;
max_nav = max_nav.max(nav_u128);
min_nav = min_nav.min(nav_u128);
nav_sum = nav_sum
    .checked_add(nav_u128)
    .ok_or(ErrorCode::MathOverflow)?;
```

```
// ...  
  
let avg = nav_sum  
  .checked_div(nav_data.len() as u128)  
  .ok_or(ErrorCode::MathOverflow)?;
```

Repeating one source gives its value additional weight in the arithmetic mean and can make a configuration appear to contain more independent corroboration than it does. Duplicate entries do not bypass the minimum-to-maximum divergence check because repetition does not change either endpoint, but they can move the final average toward the duplicated value while it remains inside the accepted range.

Only the trusted Swap administrator or Treasury manager can initialize or update the source vectors. Those roles can already choose a single source or hardcoded price, and no duplicate was identified in the assessed Swap configurations. This is therefore a configuration-hardening issue rather than a permissionless exploit.

Impact

An accidental duplicate can bias the NAV average and give operators a false impression of oracle redundancy. Transactions may execute at a value weighted differently from the intended independent-source average. The issue grants no new configuration authority and does not bypass the divergence bound.

Recommendations

Reject duplicate source identities in `validate_nav`. Compare the account and price offset for fixed-address sources, compare the account and feed identity for Pyth and Redstone, and reject repeated hardcoded entries; a pairwise check is sufficient for the five-entry limit.

Remediation

This issue has been acknowledged by Uniform Labs, and fixes were implemented in the following commits:

- [bc62fe9d](#) ↗
- [d524cdb2](#) ↗

3.18. Closing one pair requires clearing the LP's shared Treasury position

Target	SVM Swap program		
Category	Protocol Risks	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

An LP's Treasury UserPosition is derived from the LP vault-authority PDA and is shared across that LP's Treasury-backed pairs and assets. When the LP calls `close_pair`, the instruction derives this LP-wide position and requires its principal, accumulated yield, and pending-deposit count to be zero.

```
// programs/swap-program/src/instructions/close_pair.rs#L149-L175
let expected_position_key = Pubkey::find_program_address(
    &[
        b"USER_POSITION",
        ctx.accounts.lp_vault_authority.key().as_ref(),
    ],
    &treasury::ID,
)
.0;

require_keys_eq!(
    ctx.accounts.treasury_user_position.key(),
    expected_position_key,
    ErrorCode::InvalidAccountPublicKey
);

if !ctx.accounts.treasury_user_position.data_is_empty() {
    let data = ctx.accounts.treasury_user_position.try_borrow_data()?;
    let position = treasury::UserPosition::try_deserialize(&mut &data[..])?;
    require!(
        position.principal_amount() == 0
        && position.accumulated_yield() == 0
        && position.pending_deposit_count() == 0,
        ErrorCode::TreasuryBalanceNonZero
    );
}
```

Neither derivation includes the pair being closed. Balances originating from unrelated active pairs

therefore prevent the LP from closing a pair whose own liquidity has already been removed. The LP can redeem the shared balance, close the target pair, and then deposit again for its remaining routes, but this adds operational work and can briefly interrupt Treasury routing.

An unrelated user cannot trigger the behavior because only the liquidity provider can close its pair. The program API and Swap documentation also state that the Treasury balance must be zero before a pair is closed. The engagement architecture intentionally shares liquidity and the Treasury position across an LP's pairs, so the behavior is an accepted lifecycle coupling rather than an implementation deviation.

Impact

Retiring one pair may require temporarily unwinding and rebuilding Treasury exposure used by other pairs of the same LP. This creates operational overhead and a short availability interruption, but it does not cause loss, require other pairs to be closed, or permanently lock the position.

Recommendations

Document and automate the LP-wide unwind, pair closure, and redeposit sequence in LP tooling. The tooling should make the shared zero-balance requirement explicit and prevent `close_pair` from being submitted until the Treasury position is empty.

Remediation

This issue has been acknowledged by Uniform Labs.

3.19. Manager's ambient signature is exposed to asset transfer hooks

Target	treasury		
Category	Integration Risks	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

manager_withdraw, close_asset_config, and manager_deposit keep manager as a writable Signer and transfer assets via spl_token_2022::onchain::invoke_transfer_checked, forwarding remaining_accounts so the asset mint's transfer hook runs. Transaction-level signer status is preserved across CPI, so a malicious/compromised hook on a supported asset can list the manager in its ExtraAccountMetaList; since these are manager-signed instructions, the manager account is forwarded into the hook's Execute with is_signer = true. The hook can then issue CPIs using the manager's signature against any writable accounts it forces into the transaction, with no separate signature or key.

It cannot escalate within the treasury: the transfer authority is the vault_authority PDA (only the program can sign its seeds), and re-entering the treasury is blocked. Preconditions are that the manager listed an asset with an untrusted/upgradeable hook and that signer privilege propagates to the hook CPI.

Impact

Bounded to the manager's own authority,, the hook can spend the manager's signature on the manager's external token accounts or manager-authorized instructions on other programs. It cannot sign for the vault PDA or exceed the manager's existing powers, so treasury/user funds are not at risk (the manager can already withdraw those). Assets are manager-curated, keeping likelihood low.

Recommendations

Execute manager transfers in isolation, no other manager-controlled authorities or valuable manager token accounts in the same transaction.

Remediation

This issue has been acknowledged by Uniform Labs, and fixes were implemented in the following commits:

- [759d311a ↗](#)
- [0f23409d ↗](#)

4. Threat Model

As time permitted, we analyzed each instruction in the program and created a written threat model for the most critical instructions. A threat model documents the high-level functionality of a given instruction, the inputs it receives, and the accounts it operates on as well as the main checks performed on them; it gives an overview of the attack surface of the programs and of the level of control an attacker has over the inputs of critical instructions.

For brevity, system accounts and well-known program accounts have not been included in the list of accounts received by an instruction; the instructions that receive these accounts make use of Anchor types, which automatically ensure that the public key of the account is correct.

Not all instructions in the audit scope may have been modeled. The absence of a threat model in this section does not necessarily suggest that an instruction is safe.

4.1. Program: swap

Instruction: `init_global_config`

Creates the singleton `GlobalConfig` and the program-level fee vault ATA owned by `program_authority`. The signer becomes the admin. The program starts paused; the admin calls `update_global_config` to unpause it.

Input parameters

- `fee_wallet`: Pubkey: Wallet that `claim_fees` later pays out to. Stored as given; not checked against any account.
- `protocol_fees_bps`: u16: Default issuer protocol fee rate. Must be ≤ 9900 .
- `spread_protocol_fees_bps`: u16: Default spread protocol fee rate. Must be ≤ 9900 .
- `fee_token_mint`: Pubkey: Mint every LP holds to pay protocol fees. Must equal the `fee_token_mint_address` account key.

Accounts

- `global_config`: The global configuration account being created.
 - Signer: No.
 - Init: Yes (`init`; not reinitializable, fails if it already exists).
 - PDA: Yes (seeds `["global_config"]`).
 - Writable: Yes.
 - Constraints: `space = 8 + GlobalConfig::INIT_SPACE`; admin pays rent.
- `admin`: The signer that becomes the admin and pays rent.
 - Signer: Yes.
 - Init: No.
 - PDA: No.

- Writable: Yes (pays rent).
 - Constraints: None. Any signer bootstraps the config and becomes its admin.
- **program_authority**: PDA that owns the fee vault ATA.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds ["program_authority"]).
 - Writable: No.
 - Constraints: UncheckedAccount; Anchor checks only the PDA address.
- **fee_token_mint_address**: Mint of the fee token.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: InterfaceAccount<Mint> of token_program_fee; must equal the fee_token_mint parameter.
- **fee_token_account**: Program-level fee vault ATA.
 - Signer: No.
 - Init: Yes (init; not reinitializable).
 - PDA: Yes (ATA of fee_token_mint_address / program_authority).
 - Writable: Yes.
 - Constraints: associated_token::mint = fee_token_mint_address, authority = program_authority, token_program = token_program_fee; admin pays rent.
- **token_program_fee**: SPL Token or Token-2022 program for the fee mint.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: Interface<TokenInterface>.
- **associated_token_program**: Associated Token program.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: Standard program.
- **system_program**: System program.
 - Signer: No.
 - Init: No.

- PDA: No.
- Writable: No.
- Constraints: Standard program.

Additional checks and behavior

- `protocol_fees_bps <= 9900` and `spread_protocol_fees_bps <= 9900`, else `OutOfRange`.
- `fee_token_mint_address.key() == fee_token_mint`, else `InvalidAccountPublicKey`.
- Sets `paused = true`. Stores the admin (the signer), fee wallet, fee rates, fee mint, and bump.

CPI

- Associated Token program creates the fee vault ATA. System program funds rent for the created accounts.

Instruction: `set_new_admin`

Records `pending_new_admin` on `GlobalConfig` to start a two-step admin handover. The current admin signs. Emits `NewAdminProposed`.

Input parameters

- `new_admin`: Pubkey: The proposed admin. Must differ from the current admin.

Accounts

- `global_config`: The global config that receives the pending admin.
 - Signer: No.
 - Init: No.
 - PDA: Yes (`seeds ["global_config"]`, `bump = global_config.bump`).
 - Writable: Yes.
 - Constraints: `has_one = admin`.
- `admin`: The current admin.
 - Signer: Yes.

- Init: No.
- PDA: No.
- Writable: No.
- Constraints: Must equal `global_config.admin` (`has_one`).

Additional checks and behavior

- `global_config.admin != new_admin, else InvalidNewAdmin`.
- Sets `pending_new_admin = Some(new_admin)`. The pending admin completes the handover with `confirm_new_admin`.

CPI

- None.

Instruction: `confirm_new_admin`

Promotes `pending_new_admin` to `admin` and clears the pending field. The pending admin signs to accept. Emits `AdminChanged`.

Input parameters

- None.

Accounts

- **`global_config`**: The global config that receives the new admin.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds `["global_config"]`, `bump = global_config.bump`).
 - Writable: Yes.
 - Constraints: No `has_one`. The handler authorizes by matching the signer to `pending_new_admin`.
- **`new_admin`**: The pending admin accepting the role.
 - Signer: Yes.
 - Init: No.
 - PDA: No.

- Writable: No.
- Constraints: Must equal `global_config.pending_new_admin`, else Unauthorized.

Additional checks and behavior

- Returns `InvalidNewAdmin` if no pending admin is set.
- Sets `admin = new_admin`, `pending_new_admin = None`.

CPI

- None.

Instruction: `update_global_config`

Writes any subset of `GlobalConfig` fields; a `None` argument leaves that field unchanged. The admin signs. Emits `GlobalConfigUpdated`. Changing `fee_token_mint` invalidates every LP's current fee-token balance and blocks their swaps until they set up the new mint.

Input parameters

- `fee_wallet`: `Option<Pubkey>`: New fee recipient, or unchanged.
- `paused`: `Option<bool>`: New global pause state, or unchanged.
- `protocol_fees_bps`: `Option<u16>`: New default issuer fee rate (≤ 9900), or unchanged.
- `spread_protocol_fees_bps`: `Option<u16>`: New default spread fee rate (≤ 9900), or unchanged.
- `fee_token_mint`: `Option<Pubkey>`: New fee token mint, or unchanged. Stored as given; not checked against any mint account.

Accounts

- `global_config`: The global config being updated.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds `["global_config"]`, `bump = global_config.bump`).
 - Writable: Yes.
 - Constraints: `has_one = admin`.

- **admin:** The current admin.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: Must equal `global_config.admin`.

Additional checks and behavior

- Each supplied fee rate must be ≤ 9900 .
- Rewrites all fields, re-writing unchanged ones with their current values.

CPI

- None.
-

Instruction: `set_lp_protocol_fee_config`

Sets per-LP protocol fee overrides, creating the `LpProtocolFeeConfig` PDA if it does not exist. The admin signs. The `treasury_enabled` flag stays unchanged.

Input parameters

- `protocol_fees_bps`: u16: LP-specific issuer fee rate. Must be ≤ 9900 .
- `spread_protocol_fees_bps`: u16: LP-specific spread fee rate. Must be ≤ 9900 .

Accounts

- **admin:** The admin authorizing and paying for the account.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Writable: Yes (pays rent).
 - Constraints: Must equal `global_config.admin`.
- **global_config:** Backs the admin check.
 - Signer: No.

- Init: No.
- PDA: Yes (seeds ["global_config"], bump = global_config.bump).
- Writable: No.
- Constraints: has_one = admin.
- **lp_protocol_fee_config**: The per-LP fee override account.
 - Signer: No.
 - Init: Yes (init_if_needed; a second call overwrites the fee fields).
 - PDA: Yes (seeds ["lp_protocol_fee_config", liquidity_provider]).
 - Writable: Yes.
 - Constraints: space = 8 + LpProtocolFeeConfig::INIT_SPACE; admin pays rent.
- **liquidity_provider**: The LP whose overrides are set.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: UncheckedAccount; serves only as a PDA seed. No ownership or signature check.
- **system_program**: System program.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: Standard program.

Additional checks and behavior

- Both rates must be ≤ 9900 .
- Sets liquidity_provider, protocol_fees_bps, spread_protocol_fees_bps, and bump. A fresh account leaves treasury_enabled at false.

CPI

- System program creates the account when init_if_needed fires.

Instruction: `set_lp_treasury_enabled`

Turns treasury-backed liquidity routing on or off for the calling LP. On first use it creates the `LpProtocolFeeConfig` with the global default fee rates.

Input parameters

- `enabled`: `bool`: `true` opts the LP into treasury routing, `false` opts out.

Accounts

- **`liquidity_provider`**: The LP toggling its own flag and paying for account creation.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Writable: Yes (pays rent).
 - Constraints: The config PDA seeds bind to this signer, so an LP edits only its own config.
- **`global_config`**: Source of the default fee rates on first init.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds `["global_config"]`, `bump = global_config.bump`).
 - Writable: No.
 - Constraints: None beyond seeds.
- **`lp_protocol_fee_config`**: The per-LP config created or updated.
 - Signer: No.
 - Init: Yes (`init_if_needed`; a second call flips the flag).
 - PDA: Yes (seeds `["lp_protocol_fee_config", liquidity_provider]`).
 - Writable: Yes.
 - Constraints: `space = 8 + LpProtocolFeeConfig::INIT_SPACE`; `liquidity_provider` pays rent.
- **`system_program`**: System program.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: Standard program.

Additional checks and behavior

- On a fresh account (`liquidity_provider == Pubkey::default()`), sets `liquidity_provider`, `copies_protocol_fees_bps` and `spread_protocol_fees_bps` from `global_config`, and sets the bump. Then writes `treasury_enabled = enabled`.

CPI

- System program creates the account when `init_if_needed` fires.

Instruction: swap

Swaps between a pair's quote and base tokens, applying the pair fee and the protocol fee. The trader signs. When the LP enables treasury routing and the stable side is registered and unpaused in the treasury, the stable vault movements route through the treasury program. Emits `SwapExecuted`.

Input parameters

- `amount`: `u64`: Exact input (`ExactIn`) or exact output (`ExactOut`) amount per `swap_type`. Must be `> 0`.
- `min_amount_out`: `Option<u64>`: Slippage floor for `ExactIn`. Returns `AmountOutTooLow` when the output is below it.
- `max_amount_in`: `Option<u64>`: Slippage ceiling for `ExactOut`. Returns `AmountInTooHigh` when the required input is above it.
- `swap_direction`: `SwapDirection`: `QuoteToBase` (quote is input) or `BaseToQuote` (base is input).
- `swap_type`: `SwapType`: `ExactIn` or `ExactOut`.
- `treasury_nav_account_count`: `u8`: Count of NAV accounts in the trailing `remaining_accounts` for the treasury CPI.
- `treasury_apy_account_count`: `u8`: Count of APY accounts in the trailing `remaining_accounts` for the treasury CPI.

Accounts

- **user**: The trader.
 - Signer: Yes.
 - Init: No.
 - PDA: No.

- Writable: Yes.
 - Constraints: Authority of the two user token accounts.
- **pair**: The trading pair.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds ["pair", liquidity_provider, quote_mint, base_mint], bump = pair.bump).
 - Writable: No.
 - Constraints: Seeds bind it to liquidity_provider, the quote mint, and the base mint.
- **program_authority**: Signs the protocol-fee transfer into the global fee vault.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds ["program_authority"]).
 - Writable: No.
 - Constraints: UncheckedAccount; address checked by seeds.
- **liquidity_provider**: The pair's LP.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: UncheckedAccount; seeds pair, lp_vault_authority, lp_fee_vault_authority, lp_stable_config_*, and lp_protocol_fee_config.
- **lp_vault_authority**: Owns and signs the LP vault ATAs.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds ["vault_authority", liquidity_provider]).
 - Writable: Yes.
 - Constraints: UncheckedAccount; address checked by seeds. A treasury deposit can pull rent lamports from it.
- **lp_fee_vault_authority**: Owns and signs the LP protocol-fee token account.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds ["lp_fee_vault_authority", liquidity_provider]).
 - Writable: No.
 - Constraints: UncheckedAccount; address checked by seeds.
- **quote_token_mint_address**: The quote mint.

- Signer: No.
- Init: No.
- PDA: No.
- Writable: No.
- Constraints: InterfaceAccount<Mint>; used in seeds and for decimals.
- **base_token_mint_address**: The base mint.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: InterfaceAccount<Mint>; used in seeds and for decimals.
- **base_token_user_token_account**: The trader's base token account.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: Yes.
 - Constraints: token::mint = base_token_mint_address, token::authority = user, token::token_program = token_program_base.
- **quote_token_user_token_account**: The trader's quote token account.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: Yes.
 - Constraints: token::mint = quote_token_mint_address, token::authority = user, token::token_program = token_program_quote.
- **quote_token_vault_token_account**: The LP quote vault ATA.
 - Signer: No.
 - Init: No.
 - PDA: Yes (ATA of quote_token_mint_address / lp_vault_authority).
 - Writable: Yes.
 - Constraints: associated_token::authority = lp_vault_authority, matching mint and token program.
- **base_token_vault_token_account**: The LP base vault ATA.
 - Signer: No.
 - Init: No.
 - PDA: Yes (ATA of base_token_mint_address / lp_vault_authority).
 - Writable: Yes.

- Constraints: `associated_token::authority = lp_vault_authority`, matching mint and token program.
- **global_config**: Holds the pause state and the fee-mint reference.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds `["global_config"]`, bump = `global_config.bump`).
 - Writable: No.
 - Constraints: None beyond seeds.
- **fee_token_mint_address**: The fee-token mint.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: Must equal `global_config.fee_token_mint`.
- **lp_fee_token_account**: The LP protocol-fee token account, debited for the fee.
 - Signer: No.
 - Init: No.
 - PDA: Yes (ATA of `fee_token_mint_address / lp_fee_vault_authority`).
 - Writable: Yes.
 - Constraints: Matching mint, authority, and token program; balance must be $\geq$ `protocol_fees`.
- **fee_token_account**: The global fee vault, credited with the fee.
 - Signer: No.
 - Init: No.
 - PDA: Yes (ATA of `fee_token_mint_address / program_authority`).
 - Writable: Yes.
 - Constraints: Matching mint, authority, and token program.
- **base_config**: Asset config for the base token.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds `["asset", base_mint]`, bump = `base_config.bump`).
 - Writable: No.
 - Constraints: Supplies the base NAV, asset type, and pause flag.
- **quote_config**: Asset config for the quote token.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds `["asset", quote_mint]`, bump = `quote_config.bump`).
 - Writable: No.

- Constraints: Supplies the quote NAV, asset type, and pause flag.
- **fee_config**: Asset config for the fee token.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds ["asset", fee_mint], bump = fee_config.bump).
 - Writable: No.
 - Constraints: Supplies the fee-token NAV.
- **lp_stable_config_quote**: Optional per-LP stable pause config for the quote token.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds ["lp_stable_config", quote_mint, liquidity_provider]).
 - Writable: No.
 - Constraints: Option. Required and unpaused when the quote side is a Stable asset, else MissingLpStableConfig.
- **lp_stable_config_base**: Optional per-LP stable pause config for the base token.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds ["lp_stable_config", base_mint, liquidity_provider]).
 - Writable: No.
 - Constraints: Option. Required and unpaused when the base side is a Stable asset, else MissingLpStableConfig.
- **lp_protocol_fee_config**: Per-LP fee and treasury overrides.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds ["lp_protocol_fee_config", liquidity_provider]).
 - Writable: No.
 - Constraints: UncheckedAccount. When non-empty, the handler checks owner == program and a matching stored liquidity_provider, then reads the fee rates and treasury_enabled. When empty, the swap uses global rates and disables treasury.
- **token_program_quote**: Token program for the quote mint.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: Interface<TokenInterface>.
- **token_program_base**: Token program for the base mint.

- Signer: No.
- Init: No.
- PDA: No.
- Writable: No.
- Constraints: Interface<TokenInterface>.
- **token_program_fee**: Token program for the fee mint.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: Interface<TokenInterface>.

Additional checks and behavior

- Rejects when `global_config.paused`, `pair.paused`, `base_config.paused` (`RwaPaused`), or `quote_config.paused` (`StablePaused`).
- For a `Stable` side, the matching `lp_stable_config` must be present and unpaused. Omitting the optional account does not bypass the pause.
- `amount > 0`.
- Reads LP overrides only when `lp_protocol_fee_config` is initialized and its stored `liquidity_provider` matches. Otherwise it uses global rates and disables treasury.
- `pair_fee_bps` selection: `RWA` $\leftrightarrow$ `RWA` uses `redemption_fee_bps + discount_rate_bps`; `QuoteToBase` uses `redemption_fee_bps`; `BaseToQuote` uses `discount_rate_bps`.
- The base, quote, and fee-token NAV must each be non-zero.
- Requires `protocol_fees > 0` when any protocol fee bps is set, `discount_amount > 0` when `pair_fee_bps > 0`, and positive `amount_in_for_vault` and `amount_out`.
- Applies the `min_amount_out` and `max_amount_in` slippage guards when supplied.
- Transfer order: protocol fee (LP fee account $\rightarrow$ global fee vault); optional treasury redeem of the stable output into the vault; vault $\rightarrow$ user (output); user $\rightarrow$ vault (input); optional treasury deposit of the stable input.
- Per-side treasury routing also requires `asset_enabled_in_treasury`: the treasury `AssetConfig` must be treasury-owned, unpaused, and `Stable`.
- `remaining_accounts`: with treasury routing on, the first 14 are the fixed treasury bundle (program, treasury global config, quote asset config, base asset config, user position, vault authority, treasury mint, user treasury token account, quote asset vault, base asset vault, latest APY entry, token program, ATA program, system program); the rest are NAV, APY, and transfer-hook accounts. With treasury off, all `remaining_accounts` are NAV and hook accounts.

CPI

- SPL transfer_checked for the protocol fee, the output, and the input, each via the relevant mint's token program.
- treasury::cpi::redeem(ExactOut) sources a stable output before paying the user.
- treasury::cpi::deposit(ExactIn) routes a received stable input into the treasury. A System program transfer first tops up lp_vault_authority lamports for treasury account creation and realloc rent.

Instruction: init_asset_config_account

Registers an asset (RWA or Stable) with its NAV sources and creates the program fee vault ATA for that mint. The admin signs.

Input parameters

- nav_data: Vec<NavData>: 1–5 NAV source definitions, checked by validate_nav.
- price_difference_bps: u16: Maximum divergence between NAV sources. Must be <= 9900.
- asset_type: AssetType: Rwa or Stable.

Accounts

- **asset_config**: The asset configuration being created.
 - Signer: No.
 - Init: Yes (init; not reinitializable).
 - PDA: Yes (seeds ["asset", mint_address]).
 - Writable: Yes.
 - Constraints: space = 8 + AssetConfig::INIT_SPACE; admin pays rent.
- **mint_address**: The mint being registered.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: InterfaceAccount<Mint>; used as a seed. No other restriction.
- **admin**: The authorizer and rent payer.
 - Signer: Yes.
 - Init: No.

- PDA: No.
 - Writable: Yes.
 - Constraints: Must equal `global_config.admin`.
- **fee_vault_token_account**: The program fee vault ATA for this mint.
 - Signer: No.
 - Init: Yes (`init_if_needed`).
 - PDA: Yes (ATA of `mint_address / program_authority`).
 - Writable: Yes.
 - Constraints: `authority = program_authority`, matching mint and token program; admin pays rent.
- **program_authority**: Owns the fee vault ATA.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds `["program_authority"]`).
 - Writable: No.
 - Constraints: `UncheckedAccount`; address checked by seeds.
- **global_config**: Backs the admin check.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds `["global_config"]`, bump = `global_config.bump`).
 - Writable: No.
 - Constraints: `has_one = admin`.
- **token_program**: Token program for the asset mint.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: `Interface<TokenInterface>`.
- **system_program**: System program.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: Standard program.
- **associated_token_program**: Associated Token program.
 - Signer: No.
 - Init: No.
 - PDA: No.

- Writable: No.
- Constraints: Standard program.

Additional checks and behavior

- `price_difference_bps <= 9900`. `validate_nav` requires 1–5 sources with valid decimals and feeds, and requires each referenced NAV account in `remaining_accounts`.
- Sets `mint_address`, `nav_data`, `price_difference_bps`, `asset_type`, `bump`, `version = 1`, `used_in_pairs_count = 0`, and `paused = false`.

CPI

- Associated Token program and System program create the fee vault ATA when it does not exist.

Instruction: `update_asset_config_account`

Replaces an asset's NAV sources and price-difference bound. The admin signs. The asset type changes only while `used_in_pairs_count == 0`.

Input parameters

- `nav_data`: `Vec<NavData>`: Replacement NAV sources, checked by `validate_nav`.
- `price_difference_bps`: `u16`: Updated maximum divergence. Must be `<= 9900`.
- `asset_type`: `AssetType`: The new or asserted asset type.

Accounts

- **`asset_config`**: The asset configuration being updated.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds `["asset", mint_address]`, `bump = asset_config.bump`).
 - Writable: Yes.
 - Constraints: None beyond seeds.
- **`mint_address`**: The mint used as a seed.
 - Signer: No.

- Init: No.
- PDA: No.
- Writable: No.
- Constraints: InterfaceAccount<Mint>. No other restriction.
- **admin**: The authorizer.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: Must equal `global_config.admin`.
- **global_config**: Backs the admin check.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds `["global_config"]`, bump = `global_config.bump`).
 - Writable: No.
 - Constraints: `has_one = admin`.

Additional checks and behavior

- `price_difference_bps <= 9900`. `validate_nav` checks the new sources against the accounts in `remaining_accounts`.
- When `asset_type` differs from the stored type, requires `used_in_pairs_count == 0` (`AssetConfigInUse`).
- Overwrites `nav_data`, `price_difference_bps`, and `asset_type`. Leaves the pause state and counters unchanged.

CPI

- None.

Instruction: `init_pair`

Creates a quote/base pair for an LP and sets up the LP's vault ATAs, `UserVaultInfo` counters, protocol-fee token account, and stable configs. The LP signs and pays; the admin key is supplied for the `has_one` check. Increments `used_in_pairs_count` on both asset configs.

Input parameters

- **redemption_fee_bps**: u16: Fee on base→quote (selling RWA). <= 9900.
- **discount_rate_bps**: u16: Fee on quote→base (buying RWA). <= 9900, and redemption_fee_bps + discount_rate_bps <= 10_000.

Accounts

- **admin**: The authorizer, by identity only.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: UncheckedAccount; must equal global_config.admin (has_one). Does not sign.
- **liquidity_provider**: The pair owner, who signs and pays.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Writable: Yes.
 - Constraints: Pays rent for every created account.
- **pair**: The pair account being created.
 - Signer: No.
 - Init: Yes (init; not reinitializable).
 - PDA: Yes (seeds ["pair", liquidity_provider, quote_mint, base_mint]).
 - Writable: Yes.
 - Constraints: quote_token_mint_address != base_token_mint_address; LP pays rent.
- **quote_token_mint_address**: The quote mint.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: InterfaceAccount<Mint>.
- **base_token_mint_address**: The base mint.
 - Signer: No.
 - Init: No.
 - PDA: No.

- Writable: No.
 - Constraints: InterfaceAccount<Mint>.
- **lp_vault_authority**: Owns the vault ATAs.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds ["vault_authority", liquidity_provider]).
 - Writable: No.
 - Constraints: UncheckedAccount; address checked by seeds.
- **lp_fee_vault_authority**: Owns the LP fee ATA.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds ["lp_fee_vault_authority", liquidity_provider]).
 - Writable: No.
 - Constraints: UncheckedAccount; address checked by seeds.
- **quote_token_vault_token_account**: The quote vault ATA.
 - Signer: No.
 - Init: Yes (init_if_needed; shared across the LP's pairs).
 - PDA: Yes (ATA of quote_token_mint_address / lp_vault_authority).
 - Writable: Yes.
 - Constraints: Matching mint and token program; LP pays rent.
- **base_token_vault_token_account**: The base vault ATA.
 - Signer: No.
 - Init: Yes (init_if_needed; shared across the LP's pairs).
 - PDA: Yes (ATA of base_token_mint_address / lp_vault_authority).
 - Writable: Yes.
 - Constraints: Matching mint and token program; LP pays rent.
- **lp_vault_quote_pda**: UserVaultInfo counter for the quote mint.
 - Signer: No.
 - Init: Yes (init_if_needed; shared).
 - PDA: Yes (seeds ["user_vault_info", quote_mint, liquidity_provider]).
 - Writable: Yes.
 - Constraints: space = 8 + UserVaultInfo::INIT_SPACE; LP pays rent; used incremented.
- **lp_vault_base_pda**: UserVaultInfo counter for the base mint.
 - Signer: No.
 - Init: Yes (init_if_needed; shared).
 - PDA: Yes (seeds ["user_vault_info", base_mint,

- liquidity_provider])).
 - Writable: Yes.
 - Constraints: space = 8 + UserVaultInfo::INIT_SPACE; LP pays rent; used incremented.
- **global_config**: Backs the admin check and holds the pause state.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds ["global_config"], bump = global_config.bump).
 - Writable: No.
 - Constraints: has_one = admin.
- **fee_token_mint_address**: The fee-token mint.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: Must equal global_config.fee_token_mint.
- **lp_fee_token_account**: The LP protocol-fee token account.
 - Signer: No.
 - Init: Yes (init_if_needed).
 - PDA: Yes (ATA of fee_token_mint_address / lp_fee_vault_authority).
 - Writable: Yes.
 - Constraints: Matching mint and token program; LP pays rent.
- **lp_stable_config_quote**: Optional stable config for the quote token.
 - Signer: No.
 - Init: Yes (init_if_needed, Option).
 - PDA: Yes (seeds ["lp_stable_config", quote_mint, liquidity_provider])).
 - Writable: Yes.
 - Constraints: Required (Some) when the quote asset is Stable; fields set only when newly created; LP pays rent.
- **lp_stable_config_base**: Optional stable config for the base token.
 - Signer: No.
 - Init: Yes (init_if_needed, Option).
 - PDA: Yes (seeds ["lp_stable_config", base_mint, liquidity_provider])).
 - Writable: Yes.
 - Constraints: Required (Some) when the base asset is Stable; fields set only when newly created; LP pays rent.
- **quote_asset_config**: Asset config for the quote token; its counter is incremented.

- Signer: No.
- Init: No.
- PDA: Yes (seeds ["asset", quote_mint], bump = quote_asset_config.bump).
- Writable: Yes.
- Constraints: None beyond seeds.
- **base_asset_config**: Asset config for the base token; its counter is incremented.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds ["asset", base_mint], bump = base_asset_config.bump).
 - Writable: Yes.
 - Constraints: None beyond seeds.
- **token_program_quote**: Token program for the quote mint.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: Interface<TokenInterface>.
- **token_program_base**: Token program for the base mint.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: Interface<TokenInterface>.
- **token_program_fee**: Token program for the fee mint.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: Interface<TokenInterface>.
- **associated_token_program**: Associated Token program.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: Standard program.
- **system_program**: System program.

- Signer: No.
- Init: No.
- PDA: No.
- Writable: No.
- Constraints: Standard program.

Additional checks and behavior

- Rejects when `global_config.paused`.
- Requires canonical order `quote_token_mint < base_token_mint` by byte comparison, else `NonCanonicalTokenOrder`. This blocks a second pair over the same asset set.
- Fee validation: each ≤ 9900 , sum $\leq 10_000$.
- A `Stable` quote or base requires the matching `lp_stable_config_*` (`MissingLpStableConfig`). When present, the asset type must be `Stable` (`InvalidAssetType`).
- Increments used on both `UserVaultInfo` PDAs and `used_in_pairs_count` on both asset configs by 1 (unchecked add).

CPI

- Associated Token program and System program create the ATAs and PDAs.

Instruction: `update_pair`

Writes a pair's fee rates and pause flag. The pair's LP signs.

Input parameters

- `redemption_fee_bps`: u16: New base→quote fee. ≤ 9900 .
- `discount_rate_bps`: u16: New quote→base fee. ≤ 9900 , sum $\leq 10_000$.
- `paused`: bool: New pause state for this pair.

Accounts

- **`liquidity_provider`**: The pair owner.
 - Signer: Yes.
 - Init: No.
 - PDA: No.

- Writable: No.
 - Constraints: `has_one = liquidity_provider` on `pair`.
- **pair**: The pair being updated.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds `["pair", liquidity_provider, quote_mint, base_mint]`, bump = `pair.bump`).
 - Writable: Yes.
 - Constraints: `has_one = liquidity_provider`.
- **quote_token_mint_address**: The quote mint used as a seed.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: `InterfaceAccount<Mint>`.
- **base_token_mint_address**: The base mint used as a seed.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: `InterfaceAccount<Mint>`.
- **global_config**: Holds the pause state.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds `["global_config"]`, bump = `global_config.bump`).
 - Writable: No.
 - Constraints: None beyond seeds.

Additional checks and behavior

- Rejects when `global_config.paused`.
- Fee validation: each ≤ 9900 , sum $\leq 10_000$.
- Writes the new fee rates and pause flag.

CPI

- None.

Instruction: close_pair

Closes a pair and returns its rent to the LP. When a mint's `UserVaultInfo.used` reaches 0, returns the vault balance to the LP and closes the vault ATA and `UserVaultInfo` PDA. When the LP uses treasury routing, the treasury position must be empty. Drain liquidity with `remove_liquidity` first.

Input parameters

- None.

Accounts

- **admin:** The authorizer, by identity only.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: `UncheckedAccount`; must equal `global_config.admin` (`has_one`).
- **liquidity_provider:** Receives the tokens and reclaimed rent.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Writable: Yes.
 - Constraints: Destination and authority for the returned tokens and rent.
- **pair:** The pair, closed to the LP.
 - Signer: No.
 - Init: No (closed).
 - PDA: Yes (seeds `["pair", liquidity_provider, quote_mint, base_mint]`, bump = `pair.bump`).
 - Writable: Yes.
 - Constraints: `close = liquidity_provider`.
- **quote_token_mint_address:** The quote mint used as a seed.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.

- Constraints: InterfaceAccount<Mint>.
- **base_token_mint_address**: The base mint used as a seed.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: InterfaceAccount<Mint>.
- **lp_vault_authority**: Signs the vault transfers and closures.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds ["vault_authority", liquidity_provider]).
 - Writable: No.
 - Constraints: UncheckedAccount; address checked by seeds.
- **quote_token_vault_token_account**: The quote vault ATA, drained and closed when used reaches 0.
 - Signer: No.
 - Init: No.
 - PDA: Yes (ATA of quote_token_mint_address / lp_vault_authority).
 - Writable: Yes.
 - Constraints: Matching mint, authority, and token program.
- **base_token_vault_token_account**: The base vault ATA, drained and closed when used reaches 0.
 - Signer: No.
 - Init: No.
 - PDA: Yes (ATA of base_token_mint_address / lp_vault_authority).
 - Writable: Yes.
 - Constraints: Matching mint, authority, and token program.
- **lp_quote_token_account**: The LP destination for the quote token.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: Yes.
 - Constraints: token::mint = quote_token_mint_address, token::authority = liquidity_provider, matching token program.
- **lp_base_token_account**: The LP destination for the base token.
 - Signer: No.
 - Init: No.
 - PDA: No.

- Writable: Yes.
- Constraints: token::mint = base_token_mint_address,
token::authority = liquidity_provider, matching token program.
- **lp_vault_quote_pda**: UserVaultInfo for the quote mint, decremented and closed at 0.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds ["user_vault_info", quote_mint,
liquidity_provider], bump = lp_vault_quote_pda.bump).
 - Writable: Yes.
 - Constraints: None beyond seeds.
- **lp_vault_base_pda**: UserVaultInfo for the base mint, decremented and closed at 0.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds ["user_vault_info", base_mint,
liquidity_provider], bump = lp_vault_base_pda.bump).
 - Writable: Yes.
 - Constraints: None beyond seeds.
- **quote_asset_config**: Quote asset config; used_in_pairs_count decremented.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds ["asset", quote_mint], bump =
quote_asset_config.bump).
 - Writable: Yes.
 - Constraints: None beyond seeds.
- **base_asset_config**: Base asset config; used_in_pairs_count decremented.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds ["asset", base_mint], bump =
base_asset_config.bump).
 - Writable: Yes.
 - Constraints: None beyond seeds.
- **global_config**: Backs the pause and admin checks.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds ["global_config"], bump = global_config.bump).
 - Writable: No.
 - Constraints: has_one = admin.
- **lp_protocol_fee_config**: Per-LP config, read only when initialized.

- Signer: No.
- Init: No.
- PDA: Yes (seeds ["lp_protocol_fee_config", liquidity_provider]).
- Writable: No.
- Constraints: UncheckedAccount.
- **treasury_user_position**: The LP's treasury position, which must be empty.
 - Signer: No.
 - Init: No.
 - PDA: Yes (treasury-program PDA ["USER_POSITION", lp_vault_authority]).
 - Writable: No.
 - Constraints: UncheckedAccount. The handler derives the expected treasury PDA and requires the passed key to match. When the account exists, it must report zero principal, zero yield, and zero pending deposit count.
- **token_program_quote**: Token program for the quote mint.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: Interface<TokenInterface>.
- **token_program_base**: Token program for the base mint.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: Interface<TokenInterface>.

Additional checks and behavior

- Rejects when `global_config.paused`.
- Checks `treasury_user_position` against the derived treasury PDA. When non-empty, requires `principal_amount == 0 && accumulated_yield == 0 && pending_deposit_count == 0 (TreasuryBalanceNonZero)`. An empty account means the LP never deposited.
- Decrements used on both `UserVaultInfo` PDAs and `used_in_pairs_count` on both asset configs (unchecked subtraction).
- For each mint whose used reaches 0: transfers the full vault balance to the LP, closes the vault ATA, then closes the `UserVaultInfo` PDA, sending all rent to the LP.
- `remaining_accounts`: transfer-hook accounts for the vault → LP transfers.

CPI

- SPL `transfer_checked` and `close_account` move the vault balance to the LP and close the vault ATA, signed by `lp_vault_authority`. Anchor `close` closes the pair and the `UserVaultInfo` PDAs.

Instruction: `add_liquidity`

Moves amount tokens from the LP into the vault. When treasury routing is on and the asset is a treasury-registered stable, deposits the same amount into the treasury program to earn yield.

Input parameters

- `amount`: u64: Tokens to deposit. Must be > 0.
- `treasury_nav_account_count`: u8: Count of NAV accounts in `remaining_accounts` for the treasury CPI.
- `treasury_apy_account_count`: u8: Count of APY accounts in `remaining_accounts` for the treasury CPI.

Accounts

- **`liquidity_provider`**: The depositor and source authority.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Writable: Yes.
 - Constraints: Authority of `lp_token_account`. Pays rent when funding `lp_vault_authority`.
- **`mint_address`**: The token being deposited.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: `InterfaceAccount<Mint>`.
- **`lp_token_account`**: The LP source account.
 - Signer: No.
 - Init: No.
 - PDA: No.

- Writable: Yes.
- Constraints: `token::mint = mint_address, token::authority = liquidity_provider, matching token program.`
- **vault_token_account**: The vault ATA destination.
 - Signer: No.
 - Init: No.
 - PDA: Yes (ATA of `mint_address / lp_vault_authority`).
 - Writable: Yes.
 - Constraints: `associated_token::authority = lp_vault_authority, matching mint and token program.`
- **lp_vault_authority**: The vault owner; the treasury CPI can top it up with rent.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds `["vault_authority", liquidity_provider]`).
 - Writable: Yes.
 - Constraints: `UncheckedAccount`; address checked by seeds; signs the treasury deposit.
- **global_config**: Holds the pause state.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds `["global_config"]`, bump = `global_config.bump`).
 - Writable: No.
 - Constraints: None beyond seeds.
- **lp_protocol_fee_config**: Source of the treasury toggle.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds `["lp_protocol_fee_config", liquidity_provider]`).
 - Writable: No.
 - Constraints: `UncheckedAccount`. When non-empty, the handler checks `owner == program` and a matching `liquidity_provider`.
- **treasury_program**: The treasury program.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: `Option<UncheckedAccount>`. With treasury routing on, checked equal to `treasury::ID`.
- **treasury_global_config**: Treasury global config.

- Signer: No.
 - Init: No.
 - PDA: Yes (treasury-side).
 - Writable: No.
 - Constraints: Option<UncheckedAccount>; the treasury CPI validates it.
- **treasury_asset_config**: Treasury asset config.
 - Signer: No.
 - Init: No.
 - PDA: Yes (treasury-side).
 - Writable: No.
 - Constraints: Option<UncheckedAccount>. asset_enabled_in_treasury reads it; the treasury CPI validates it.
- **treasury_user_position**: Treasury user position.
 - Signer: No.
 - Init: No (the treasury CPI creates it when needed).
 - PDA: Yes (treasury-side).
 - Writable: Yes.
 - Constraints: Option<UncheckedAccount>; the treasury CPI validates it.
- **treasury_vault_authority**: Treasury vault authority.
 - Signer: No.
 - Init: No.
 - PDA: Yes (treasury-side).
 - Writable: No.
 - Constraints: Option<UncheckedAccount>; the treasury CPI validates it.
- **treasury_mint**: Treasury receipt mint.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: Yes.
 - Constraints: Option<UncheckedAccount>; the treasury CPI validates it.
- **treasury_asset_vault**: Treasury asset vault.
 - Signer: No.
 - Init: No.
 - PDA: Yes (treasury-side).
 - Writable: Yes.
 - Constraints: Option<UncheckedAccount>; the treasury CPI validates it.
- **treasury_user_treasury_token_account**: The LP's treasury-token account.
 - Signer: No.

- Init: No (the treasury CPI creates it when needed).
 - PDA: No.
 - Writable: Yes.
 - Constraints: Option<UncheckedAccount>; the treasury CPI validates it.
- **treasury_token_program**: Token program for the treasury mint.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: Option<UncheckedAccount>; the treasury CPI validates it.
- **treasury_associated_token_program**: Associated Token program for the treasury CPI.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: Option<UncheckedAccount>.
- **treasury_system_program**: System program for the treasury CPI.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: Option<UncheckedAccount>; checked equal to the System program when funding rent.
- **token_program**: Token program for mint_address.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: Interface<TokenInterface>.

Additional checks and behavior

- Rejects when global_config.paused. Requires amount > 0.
- Reads treasury_enabled from lp_protocol_fee_config. asset_enabled_in_treasury requires the treasury asset config to be treasury-owned, unpaused, and Stable. It skips RWAs.
- Always transfers amount from the LP to the vault. When both treasury conditions hold, it also CPIs a treasury deposit of the same amount (ExactIn) after topping up

lp_vault_authority lamports.

- remaining_accounts: NAV, APY, and transfer-hook accounts for the treasury deposit, split by split_remaining_slices. With treasury off, all count as transfer-hook accounts.

CPI

- SPL transfer_checked moves tokens LP → vault. treasury::cpi::deposit (ExactIn) runs when treasury is on. A System program transfer funds lp_vault_authority rent.

Instruction: remove_liquidity

Moves tokens from the vault back to the LP. When treasury routing is on for a treasury-registered stable, redeems from the treasury into the vault first and forwards the vault delta.

Input parameters

- amount: u64: Must be > 0. For a treasury-backed asset it is treasury receipt tokens to burn (ExactIn) or asset tokens to receive (ExactOut). Otherwise it is asset tokens.
- treasury_nav_account_count: u8: Count of NAV accounts for the treasury CPI.
- treasury_apy_account_count: u8: Count of APY accounts for the treasury CPI.
- exact_mode: treasury::ExactMode: ExactIn burns amount receipt tokens and lets NAV set the output; use it to drain a position fully. ExactOut sends amount asset tokens out.

Accounts

- **liquidity_provider:** The withdrawer and destination authority.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: Authority of lp_token_account.
- **mint_address:** The token being withdrawn.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.

- Constraints: InterfaceAccount<Mint>.
- **lp_token_account**: The LP destination account.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: Yes.
 - Constraints: token::mint = mint_address, token::authority = liquidity_provider, matching token program.
- **vault_token_account**: The vault ATA source, reloaded to measure the treasury delta.
 - Signer: No.
 - Init: No.
 - PDA: Yes (ATA of mint_address / lp_vault_authority).
 - Writable: Yes.
 - Constraints: associated_token::authority = lp_vault_authority, matching mint and token program.
- **lp_vault_authority**: Signs the vault and treasury transfers.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds ["vault_authority", liquidity_provider]).
 - Writable: Yes.
 - Constraints: UncheckedAccount; address checked by seeds.
- **global_config**: Holds the pause state.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds ["global_config"], bump = global_config.bump).
 - Writable: No.
 - Constraints: None beyond seeds.
- **lp_protocol_fee_config**: Source of the treasury toggle.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds ["lp_protocol_fee_config", liquidity_provider]).
 - Writable: No.
 - Constraints: UncheckedAccount; validated when non-empty.
- **treasury_program**: The treasury program.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.

- Constraints: Option<UncheckedAccount>. With treasury routing on, checked equal to treasury::ID.
- **treasury_global_config**: Treasury global config.
 - Signer: No.
 - Init: No.
 - PDA: Yes (treasury-side).
 - Writable: No.
 - Constraints: Option<UncheckedAccount>; the treasury CPI validates it.
- **treasury_asset_config**: Treasury asset config.
 - Signer: No.
 - Init: No.
 - PDA: Yes (treasury-side).
 - Writable: No.
 - Constraints: Option<UncheckedAccount>. asset_enabled_in_treasury reads it; the treasury CPI validates it.
- **treasury_user_position**: Treasury user position.
 - Signer: No.
 - Init: No.
 - PDA: Yes (treasury-side).
 - Writable: Yes.
 - Constraints: Option<UncheckedAccount>; the treasury CPI validates it.
- **treasury_vault_authority**: Treasury vault authority.
 - Signer: No.
 - Init: No.
 - PDA: Yes (treasury-side).
 - Writable: No.
 - Constraints: Option<UncheckedAccount>; the treasury CPI validates it.
- **treasury_mint**: Treasury receipt mint.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: Yes.
 - Constraints: Option<UncheckedAccount>; the treasury CPI validates it.
- **treasury_user_treasury_token_account**: The LP's treasury-token account.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: Yes.

- Constraints: `Option<UncheckedAccount>`; the treasury CPI validates it.
- **treasury_asset_vault**: Treasury asset vault.
 - Signer: No.
 - Init: No.
 - PDA: Yes (treasury-side).
 - Writable: Yes.
 - Constraints: `Option<UncheckedAccount>`; the treasury CPI validates it.
- **treasury_latest_apy_entry**: Treasury latest APY entry.
 - Signer: No.
 - Init: No.
 - PDA: Yes (treasury-side).
 - Writable: No.
 - Constraints: `Option<UncheckedAccount>`; the treasury CPI validates it.
- **treasury_token_program**: Token program for the treasury mint.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: `Option<UncheckedAccount>`; the treasury CPI validates it.
- **treasury_associated_token_program**: Associated Token program for the treasury CPI.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: `Option<UncheckedAccount>`.
- **token_program**: Token program for `mint_address`.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: `Interface<TokenInterface>`.

Additional checks and behavior

- Rejects when `global_config.paused`. Requires `amount > 0`.
- Treasury path: reads the vault balance before the redeem CPI, calls `treasury::cpi::redeem` with `exact_mode`, reloads the vault, and sets

- transfer_amount = new_balance - old_balance, which must be > 0.
- Non-treasury path: requires amount <= vault balance and uses amount directly.
- Transfers transfer_amount from the vault to the LP.
- remaining_accounts: NAV, APY, and transfer-hook accounts for the treasury redeem. With treasury off, all count as transfer-hook accounts.

CPI

- treasury::cpi::redeem runs on the treasury path. SPL transfer_checked moves the tokens vault → LP, signed by lp_vault_authority.

Instruction: set_paused_for_lp_stable_config

Sets the pause flag on one LP's stable-mint config. The LP or the admin signs.

Input parameters

- paused: bool: New pause state for this LP stable config.

Accounts

- **global_config**: Backs the program-pause check and holds the admin key.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds ["global_config"], bump = global_config.bump).
 - Writable: No.
 - Constraints: None beyond seeds.
- **lp_stable_config**: The config being updated.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds ["lp_stable_config", mint_address, liquidity_provider], bump = lp_stable_config.bump).
 - Writable: Yes.
 - Constraints: None beyond seeds.
- **mint_address**: The stable mint used as a seed.
 - Signer: No.
 - Init: No.

- PDA: No.
- Writable: No.
- Constraints: UncheckedAccount; validated through the lp_stable_config seeds.
- **liquidity_provider**: The LP used as a seed.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: UncheckedAccount; validated through the seeds.
- **signer**: The caller.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: The handler requires `signer == liquidity_provider` or `signer == global_config.admin`, else Unauthorized.

Additional checks and behavior

- Rejects when `global_config.paused`.
- Authorizes the signer as the LP or the admin.
- Writes `paused`.

CPI

- None.

Instruction: `set_paused_for_asset`

Sets the pause flag on an asset across all pairs. The admin signs. Emits `AssetPauseChanged`.

Input parameters

- `paused`: `bool`: New pause state for the asset.

Accounts

- **global_config:** Backs the admin check.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds ["global_config"], bump = global_config.bump).
 - Writable: No.
 - Constraints: has_one = admin.
- **asset_config:** The asset config being updated.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds ["asset", mint_address], bump = asset_config.bump).
 - Writable: Yes.
 - Constraints: None beyond seeds.
- **mint_address:** The asset mint used as a seed and event field.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: UncheckedAccount; validated through the asset_config seeds.
- **admin:** The authorizer.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Writable: Yes.
 - Constraints: Must equal global_config.admin.

Additional checks and behavior

- Writes asset_config.paused = paused. No program-pause gate.

CPI

- None.
-

Instruction: `claim_fees`

Sends the full program fee vault balance to `global_config.fee_wallet`. Anyone calls it; the funds reach only the configured fee wallet.

Input parameters

- `fee_wallet`: Pubkey: The fee wallet. Bound with `#[instruction]` for the `has_one = fee_wallet` constraint and as the destination account's authority.

Accounts

- **`fee_mint`**: The fee token mint.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: `InterfaceAccount<Mint>`.
- **`fee_wallet_token_account`**: The destination, owned by the fee wallet.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: Yes.
 - Constraints: `token::mint = fee_mint, token::authority = fee_wallet`, matching token program.
- **`program_authority`**: Signs the transfer.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds `["program_authority"]`).
 - Writable: No.
 - Constraints: `UncheckedAccount`; address checked by seeds.
- **`fee_token_account`**: The global fee vault source.
 - Signer: No.
 - Init: No.
 - PDA: Yes (ATA of `fee_mint / program_authority`).
 - Writable: Yes.
 - Constraints: Matching mint, authority, and token program.
- **`global_config`**: Backs the pause and fee-wallet checks.
 - Signer: No.

- Init: No.
- PDA: Yes (seeds ["global_config"], bump = global_config.bump).
- Writable: No.
- Constraints: has_one = fee_wallet. The fee_wallet argument must equal global_config.fee_wallet.
- **token_program**: Token program for the fee mint.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: Interface<TokenInterface>.

Additional checks and behavior

- Rejects when global_config.paused.
- Transfers the full fee_token_account.amount to fee_wallet_token_account when it is > 0. The only caller check is the has_one-bound destination.
- remaining_accounts: transfer-hook accounts for the transfer.

CPI

- SPL transfer_checked, signed by program_authority.

Instruction: close_lp_fee_account

Returns the LP fee token account balance to the LP and closes the account, refunding rent. Call it after closing all of the LP's pairs.

Input parameters

- None.

Accounts

- **liquidity_provider**: Receives the balance and reclaimed rent.
 - Signer: Yes.
 - Init: No.

- PDA: No.
 - Writable: Yes.
 - Constraints: Authority of `lp_destination_token_account`.
- **lp_fee_vault_authority**: Owns and signs the fee account.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds `["lp_fee_vault_authority", liquidity_provider]`).
 - Writable: No.
 - Constraints: `UncheckedAccount`; address checked by seeds.
- **global_config**: Backs the pause check and holds the fee-mint reference.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds `["global_config"]`, bump = `global_config.bump`).
 - Writable: No.
 - Constraints: None beyond seeds.
- **fee_token_mint_address**: The fee mint.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: Must equal `global_config.fee_token_mint`.
- **lp_fee_token_account**: The fee account to drain and close.
 - Signer: No.
 - Init: No.
 - PDA: Yes (ATA of `fee_token_mint_address / lp_fee_vault_authority`).
 - Writable: Yes.
 - Constraints: Matching mint, authority, and token program.
- **lp_destination_token_account**: The LP account that receives the remaining balance.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: Yes.
 - Constraints: `token::mint = fee_token_mint_address`, `token::authority = liquidity_provider`, matching token program.
- **token_program_fee**: Token program for the fee mint.
 - Signer: No.
 - Init: No.
 - PDA: No.

- Writable: No.
- Constraints: `Interface<TokenInterface>`.

Additional checks and behavior

- Rejects when `global_config.paused`.
- Transfers a non-zero balance to `lp_destination_token_account`, then closes the fee ATA and sends the rent to the LP.
- `remaining_accounts`: transfer-hook accounts for the transfer.

CPI

- SPL `transfer_checked` and `close_account`, signed by `lp_fee_vault_authority`.

4.2. Program: treasury

Instruction: initialize

Creates the singleton `GlobalConfig`, takes over the pre-made Token-2022 treasury mint, and moves the mint authority to the `vault_authority` PDA. Only the program upgrade authority signs. The treasury starts unpaused and the yield index starts at 10^{18} .

Input parameters

- `manager`: Pubkey: The operational manager stored in the config.
- `oracle_authority`: Pubkey: The key allowed to post APY through `post_daily_rate`.
- `apy_history_retention_duration`: u64: Minimum age before `prune_apy_entry` can close an APY entry. Must be greater than seven days (604800).
- `management_fee_bps`: u16: Management fee taken from posted APY. Must be ≤ 9900 .
- `max_unposted_yield_seconds`: u64: Cap on how far the latest posted APY extrapolates during redeem. Must be $\leq$ i64: `:MAX`.

Accounts

- **caller**: The signer, which must be the program upgrade authority.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Writable: Yes (pays rent).

- Constraints: Holds the current mint authority of `treasury_mint`.
- **program_data**: The program's ProgramData account, read to authorize the caller.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds [`crate::ID`] under `bpf_loader_upgradeable`).
 - Writable: No.
 - Constraints: `upgrade_authority_address == Some(caller.key()), else UnauthorizedInitializer`.
- **global_config**: The global configuration account being created.
 - Signer: No.
 - Init: Yes (`init`; not reinitializable).
 - PDA: Yes (seeds [`"GLOBAL_CONFIG"`]).
 - Writable: Yes.
 - Constraints: `space = GlobalConfig::INIT_SPACE + 8; caller pays rent`.
- **vault_authority**: PDA that receives the mint authority and later owns the asset vaults.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds [`"VAULT_AUTHORITY"`]).
 - Writable: No.
 - Constraints: `UncheckedAccount`; address checked by seeds.
- **token_program**: The Token-2022 program.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: `address == token_2022::ID, else InvalidTreasuryMintTokenProgram`.
- **treasury_mint**: The receipt-token mint the program takes over.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: Yes (mint authority is reassigned).
 - Constraints: `mint::token_program = token_program; metadata_pointer::metadata_address == treasury_mint`. Handler also checks `decimals <= 18`, supply 0, current mint authority equal to caller, no freeze authority, and non-empty metadata name and symbol.
- **system_program**: System program.
 - Signer: No.

- Init: No.
- PDA: No.
- Writable: No.
- Constraints: Standard program.

Additional checks and behavior

- `management_fee_bps <= 9900`, else `BpsValueOutOfRange`.
- `apy_history_retention_duration > 604800` (seven days), else `ApyHistoryRetentionDurationTooShort`.
- `max_unposted_yield_seconds <= i64::MAX`, else `MaxUnpostedYieldSecondsTooLarge`.
- Validates the mint (decimals, zero supply, caller-held mint authority, no freeze authority, metadata) before use.
- Sets `global_yield_index_fp = 10^18`, `last_yield_update_ts = get_day_start_ts(now)`, `latest_apy_entry = Pubkey::default()`, `max_apy_bps = 10_000`, `supported_asset_count = 0`, `paused = false`. Stores manager, oracle authority, treasury mint, retention, fee, and the extrapolation cap. Emits `InitializeEvent`.

CPI

- Token-2022 `set_authority` moves the mint authority from caller to `vault_authority`.

Instruction: `initialize_asset_config`

Registers a supported asset, creates its vault ATA owned by `vault_authority`, and increments `supported_asset_count`. The manager signs. The asset starts unpaused.

Input parameters

- `asset_type`: `AssetType`: `Rwa` or `Stable`. Controls pending-deposit maturation timing.
- `price_difference_bps`: `u16`: Maximum divergence between NAV sources. Must be `<= 9900`.
- `nav_data`: `Vec<NavData>`: 1–5 NAV source definitions, checked by `validate_nav`.

Accounts

- **manager**: The authorizer and rent payer.

- Signer: Yes.
 - Init: No.
 - PDA: No.
 - Writable: Yes (pays rent).
 - Constraints: Must equal `global_config.manager`.
- **global_config**: Backs the manager check and holds the asset counter.
 - Signer: No.
 - Init: No.
 - PDA: Yes (singleton ["GLOBAL_CONFIG"]; seeds not re-derived here).
 - Writable: Yes.
 - Constraints: `has_one = manager`.
- **asset_config**: The asset configuration being created.
 - Signer: No.
 - Init: Yes (`init`; not reinitializable).
 - PDA: Yes (seeds ["ASSET_CONFIG", `asset_mint`]).
 - Writable: Yes.
 - Constraints: `space = 8 + AssetConfig::INIT_SPACE`; manager pays rent.
- **vault_authority**: Owns the asset vault ATA.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds ["VAULT_AUTHORITY"]).
 - Writable: No.
 - Constraints: `UncheckedAccount`; address checked by seeds.
- **asset_vault**: The vault ATA for this asset.
 - Signer: No.
 - Init: Yes (`init_if_needed`).
 - PDA: Yes (ATA of `asset_mint` / `vault_authority`).
 - Writable: Yes.
 - Constraints: `associated_token::authority = vault_authority`, matching mint and token program; manager pays rent.
- **asset_mint**: The mint being registered.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: `mint::token_program = token_program`; `asset_mint != global_config.treasury_mint`, else `InvalidAssetMint`.
- **token_program**: Token program for the asset mint.

- Signer: No.
- Init: No.
- PDA: No.
- Writable: No.
- Constraints: Interface<TokenInterface>.
- **associated_token_program**: Associated Token program.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: Standard program.
- **system_program**: System program.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: Standard program.

Additional checks and behavior

- `price_difference_bps` ≤ 9900 , else `BpsValueOutOfRange`.
- `validate_nav` requires 1–5 sources with valid decimals and feeds, and requires each referenced NAV account in `remaining_accounts`.
- Sets `asset_mint`, `asset_type`, `paused` = `false`, `nav_data`, `price_difference_bps`, and `bump`. Increments `global_config.supported_asset_count` by 1 (checked add). Emits the `asset-config` event.

CPI

- Associated Token program creates the asset vault ATA when it does not exist.

Instruction: `deposit`

Moves an asset into the vault and mints treasury tokens to the user. The user signs. Before minting, it settles the user's yield and merges matured pending deposits, then records the new deposit as pending.

Input parameters

- **amount**: u64: Asset input (ExactIn) or treasury output (ExactOut), per `exact_mode`. Must be > 0.
- **nav_account_count**: u8: Count of NAV accounts at the front of `remaining_accounts`.
- **apy_account_count**: u8: Count of APY accounts after the NAV accounts.
- **exact_mode**: `ExactMode`: `ExactIn` reads amount as asset input; `ExactOut` reads it as treasury output.

Accounts

- **user**: The depositor and rent payer.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Writable: Yes.
 - Constraints: Authority of `user_asset_token_account` and `user_treasury_token_account`.
- **global_config**: Holds the pause state, yield index, and treasury mint.
 - Signer: No.
 - Init: No.
 - PDA: Yes (singleton ["GLOBAL_CONFIG"]; seeds not re-derived here).
 - Writable: No.
 - Constraints: None beyond type.
- **asset_config**: Config for the deposited asset.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds ["ASSET_CONFIG", `asset_mint`], `bump = asset_config.bump`).
 - Writable: No.
 - Constraints: Supplies NAV, asset type, and pause flag.
- **user_position**: The user's position.
 - Signer: No.
 - Init: Yes (`init_if_needed`; initialized once, guarded by a discriminator check so later deposits keep existing data).
 - PDA: Yes (seeds ["USER_POSITION", `user`]).
 - Writable: Yes.
 - Constraints: `space = UserPosition::space(DEFAULT_PENDING_CAPACITY)`; user pays rent and

any realloc.

- **vault_authority**: Signs the treasury mint.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds ["VAULT_AUTHORITY"]).
 - Writable: No.
 - Constraints: UncheckedAccount; address checked by seeds.
- **treasury_mint**: The receipt mint.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: Yes (supply increases).
 - Constraints: address == global_config.treasury_mint, mint::token_program = treasury_token_program.
- **asset_mint**: The deposited asset mint.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: address == asset_config.asset_mint, mint::token_program = asset_token_program.
- **user_asset_token_account**: The user's asset source.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: Yes.
 - Constraints: token::mint = asset_mint, token::authority = user, matching token program.
- **asset_vault**: The vault ATA destination.
 - Signer: No.
 - Init: No.
 - PDA: Yes (ATA of asset_mint / vault_authority).
 - Writable: Yes.
 - Constraints: associated_token::authority = vault_authority, matching mint and token program.
- **user_treasury_token_account**: The user's treasury receipt account.
 - Signer: No.
 - Init: Yes (init_if_needed).

- PDA: Yes (ATA of treasury_mint / user).
 - Writable: Yes.
 - Constraints: associated_token::authority = user, matching mint and token program; user pays rent.
- **asset_token_program**: Token program for the asset mint.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: Interface<TokenInterface>.
- **treasury_token_program**: Token program for the treasury mint.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: Interface<TokenInterface>.
- **associated_token_program**: Associated Token program.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: Standard program.
- **system_program**: System program.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: Standard program.

Additional checks and behavior

- Rejects when global_config.paused (ProgramPaused) or asset_config.paused (AssetPaused). Requires amount > 0.
- Reads NAV from the asset config, which must be > 0 (InvalidNav).
- Settles the position against posted yield and merges matured pending deposits before recording the new deposit.
- ExactIn: converts amount asset to value, then to a treasury amount. ExactOut: takes amount as the treasury amount and rounds the asset input up.
- Requires deposit_value_fp >= 10¹⁸ (\$1) (DepositValueBelowMinimum), and positive

treasury and asset amounts.

- Transfers the asset to the vault, mints the treasury amount, grows the position when the pending array is full, and appends the new pending deposit. Emits `DepositEvent` and, when merges occur, `PendingDepositsMergedEvent`.
- `remaining_accounts`: NAV accounts, then APY accounts, then transfer-hook accounts.

CPI

- SPL `transfer_checked` moves the asset `user` → vault. Token `mint_to` mints the treasury amount, signed by `vault_authority`. System program funds any position realloc.

Instruction: redeem

Burns treasury tokens and sends the chosen asset to the user. The user signs. It settles yield, consumes un-matured pending deposits at par newest-first, then draws settled principal and yield proportionally.

Input parameters

- `amount`: `u64`: Treasury input (`ExactIn`) or asset output (`ExactOut`), per `exact_mode`. Must be `> 0`.
- `nav_account_count`: `u8`: Count of NAV accounts at the front of `remaining_accounts`.
- `apy_account_count`: `u8`: Count of APY accounts after the NAV accounts.
- `exact_mode`: `ExactMode`: `ExactIn` reads amount as treasury input; `ExactOut` reads it as asset output.

Accounts

- **`user`**: The redeemer.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Writable: Yes.
 - Constraints: Authority of `user_treasury_token_account` and `user_asset_token_account`.
- **`global_config`**: Holds the pause state, yield index, and extrapolation cap.
 - Signer: No.
 - Init: No.

- PDA: Yes (singleton ["GLOBAL_CONFIG"]; seeds not re-derived here).
 - Writable: No.
 - Constraints: None beyond type.
- **asset_config**: Config for the redeemed asset.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds ["ASSET_CONFIG", asset_mint], bump = asset_config.bump).
 - Writable: No.
 - Constraints: Supplies NAV, asset type, and pause flag.
- **user_position**: The user's position.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds ["USER_POSITION", user]).
 - Writable: Yes.
 - Constraints: user_position.owner == user.
- **vault_authority**: Signs the asset transfer out.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds ["VAULT_AUTHORITY"]).
 - Writable: No.
 - Constraints: UncheckedAccount; address checked by seeds.
- **treasury_mint**: The receipt mint.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: Yes (supply decreases on burn).
 - Constraints: address == global_config.treasury_mint, mint::token_program = treasury_token_program.
- **user_treasury_token_account**: The user's treasury account, burned from.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: Yes.
 - Constraints: token::mint = treasury_mint, token::authority = user, matching token program.
- **asset_mint**: The redeemed asset mint.
 - Signer: No.

- Init: No.
- PDA: No.
- Writable: No.
- Constraints: `address == asset_config.asset_mint,`
`mint::token_program = asset_token_program.`
- **asset_vault:** The vault ATA source.
 - Signer: No.
 - Init: No.
 - PDA: Yes (ATA of `asset_mint / vault_authority`).
 - Writable: Yes.
 - Constraints: `associated_token::authority = vault_authority,`
`matching mint and token program.`
- **user_asset_token_account:** The user's asset destination.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: Yes.
 - Constraints: `token::mint = asset_mint, token::authority = user,`
`matching token program.`
- **latest_apy_entry:** The most recent APY entry, used for partial-day accrual.
 - Signer: No.
 - Init: No.
 - PDA: Yes (an `["APY_ENTRY", day]` account).
 - Writable: No.
 - Constraints: `latest_apy_entry.key() ==`
`global_config.latest_apy_entry, else InvalidLatestApyEntry.`
- **treasury_token_program:** Token program for the treasury mint.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: `Interface<TokenInterface>.`
- **asset_token_program:** Token program for the asset mint.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: `Interface<TokenInterface>.`
- **associated_token_program:** Associated Token program.

- Signer: No.
- Init: No.
- PDA: No.
- Writable: No.
- Constraints: Standard program.

Additional checks and behavior

- Rejects when `global_config.paused` (`ProgramPaused`) or `asset_config.paused` (`AssetPaused`). Requires `amount > 0`.
- Reads NAV, which must be `> 0`. Computes the treasury amount and asset amount from `exact_mode`, both `> 0`.
- Extends the yield index from the last posted day at the latest posted rate up to `min(now, last_yield_update_ts + max_unposted_yield_seconds)`. Past that deadline the index is frozen at the cap.
- Loads the APY entries and adds `latest_apy_entry`. Requires `latest_apy_entry` not to appear in the APY accounts (`DuplicateLatestApyEntry`).
- Consumes un-matured pending deposits at par newest-first, then draws the remainder from settled principal and yield in proportion to their balances. Requires the remainder `<= settled_total` (`InsufficientRedeemableBalance`).
- Burns `treasury_amount - consumed_yield` (yield carries no minted tokens). Writes the position, burns when the amount is `> 0`, and transfers the asset out. Emits `RedeemEvent` and, when merges occur, `PendingDepositsMergedEvent`.
- `remaining_accounts`: NAV accounts, then APY accounts, then transfer-hook accounts.

CPI

- Token burn reduces the user's treasury balance. SPL `transfer_checked` moves the asset vault $\rightarrow$ user, signed by `vault_authority`.

Instruction: `set_pending_manager`

Records `pending_manager` on `GlobalConfig` to start a two-step manager handover. The current manager signs. Emits the nomination event.

Input parameters

- None.

Accounts

- **manager**: The current manager.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: Must equal `global_config.manager` (`has_one`).
- **global_config**: Receives the pending manager.
 - Signer: No.
 - Init: No.
 - PDA: Yes (singleton ["GLOBAL_CONFIG"]; seeds not re-derived here).
 - Writable: Yes.
 - Constraints: `has_one = manager`.
- **pending_manager**: The proposed manager.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: `UncheckedAccount`; stored as `pending_manager`. No further check.

Additional checks and behavior

- Sets `pending_manager = Some(pending_manager.key())`. The pending manager completes the handover with `accept_manager`.

CPI

- None.
-

Instruction: `accept_manager`

Promotes `pending_manager` to `manager` and clears the pending field. The pending manager signs. Emits the accept event.

Input parameters

- None.

Accounts

- **pending_manager**: The pending manager accepting the role.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: Must equal `global_config.pending_manager`.
- **global_config**: Receives the new manager.
 - Signer: No.
 - Init: No.
 - PDA: Yes (singleton ["GLOBAL_CONFIG"]; seeds not re-derived here).
 - Writable: Yes.
 - Constraints: `pending_manager == Some(pending_manager.key())`.

Additional checks and behavior

- Sets `manager = pending_manager, pending_manager = None`.

CPI

- None.

Instruction: update_global_config

Writes any subset of global settings; a `None` argument leaves that field unchanged. The manager signs. Emits the update event.

Input parameters

- **oracle_authority**: `Option<Pubkey>`: New oracle authority, or unchanged.
- **apy_history_retention_duration**: `Option<u64>`: New retention. Must be greater than seven days (604800).

- `management_fee_bps`: Option<u16>: New fee. Must be ≤ 9900 and $\leq \text{max_apy_bps}$.
- `pause`: Option<bool>: New global pause state, or unchanged.
- `max_unposted_yield_seconds`: Option<u64>: New extrapolation cap. Must be $\leq i64::\text{MAX}$.

Accounts

- **manager**: The current manager.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: Must equal `global_config.manager` (has_one).
- **global_config**: The config being updated.
 - Signer: No.
 - Init: No.
 - PDA: Yes (singleton ["GLOBAL_CONFIG"]; seeds not re-derived here).
 - Writable: Yes.
 - Constraints: `has_one = manager`.

Additional checks and behavior

- `apy_history_retention_duration > 604800`, else `ApyHistoryRetentionDurationTooShort`.
- `management_fee_bps ≤ 9900` (`BpsValueOutOfRange`) and $\leq \text{max_apy_bps}$ (`MaxApyBelowManagementFee`).
- `max_unposted_yield_seconds ≤ i64::MAX`, else `MaxUnpostedYieldSecondsTooLarge`.
- Writes each supplied field and emits the requested changes.

CPI

- None.

Instruction: `set_max_apy_bps`

Sets the maximum gross APY the oracle can post per day. The manager signs. Emits the event.

Input parameters

- `max_apy_bps`: u32: New cap. Must be $\leq 10_000$ and $\geq \text{management_fee_bps}$.

Accounts

- `manager`: The current manager.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: Must equal `global_config.manager` (has_one).
- `global_config`: The config being updated.
 - Signer: No.
 - Init: No.
 - PDA: Yes (singleton ["GLOBAL_CONFIG"]; seeds not re-derived here).
 - Writable: Yes.
 - Constraints: `has_one = manager`.

Additional checks and behavior

- `max_apy_bps` $\leq 10_000$, else `BpsValueOutOfRange`.
- `max_apy_bps` $\geq \text{management_fee_bps}$, else `MaxApyBelowManagementFee`.
- Writes `max_apy_bps`.

CPI

- None.
-

Instruction: `admin_update_user_yield`

Overrides a user's accumulated yield and refreshes their credit index to the current global index. The manager signs. Emits the event.

Input parameters

- `accumulated_yield`: `Option<u64>`: `Some(v)` sets the position's accumulated yield to `v`; `None` keeps it. Either way the credit index is refreshed.

Accounts

- **manager**: The current manager.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: Must equal `global_config.manager` (`has_one`).
- **global_config**: Source of the current yield index.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds `["GLOBAL_CONFIG"]`, `bump = global_config.bump`).
 - Writable: No.
 - Constraints: `has_one = manager`.
- **user_position**: Any user's position to update.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: Yes.
 - Constraints: `AccountLoader<UserPosition>`. No seed or owner check; the manager can target any position.

Additional checks and behavior

- When `accumulated_yield` is `Some`, writes it. Always sets `global_yield_index_on_last_update` to the current global index.

CPI

- None.
-

Instruction: `set_asset_config`

Updates an asset's type, pause flag, NAV sources, or price-difference bound. The manager signs. Emits the update event with whether NAV changed.

Input parameters

- `asset_type`: `Option<AssetType>`: New asset type, or unchanged.
- `pause`: `Option<bool>`: New asset pause state, or unchanged.
- `nav_data`: `Option<Vec<NavData>>`: Replacement NAV sources (checked by `validate_nav`), or unchanged.
- `price_difference_bps`: `Option<u16>`: New maximum divergence (≤ 9900), or unchanged.

Accounts

- **manager**: The current manager.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: Must equal `global_config.manager` (`has_one`).
- **global_config**: Backs the manager check.
 - Signer: No.
 - Init: No.
 - PDA: Yes (singleton ["GLOBAL_CONFIG"]; seeds not re-derived here).
 - Writable: No.
 - Constraints: `has_one = manager`.
- **asset_config**: The asset config being updated.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds ["ASSET_CONFIG", `asset_mint`], `bump = asset_config.bump`).
 - Writable: Yes.
 - Constraints: None beyond seeds.
- **asset_mint**: The asset mint used as a seed.
 - Signer: No.
 - Init: No.
 - PDA: No.

- Writable: No.
- Constraints: `mint::token_program = token_program`.
- **token_program**: Token program for the asset mint.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: `Interface<TokenInterface>`.

Additional checks and behavior

- When `nav_data` is `Some`, runs `validate_nav` against `remaining_accounts` before writing it.
- When `price_difference_bps` is `Some`, requires `<= 9900 (BpsValueOutOfRange)`.
- Writes each supplied field. Emits `SetAssetConfigEvent` with `nav_data_updated`.

CPI

- None.

Instruction: `post_daily_rate`

Posts a day's APY or corrects a past day, then advances the global yield index. The oracle authority signs. Days post in strict chronological order.

Input parameters

- `day_start_ts`: `u64`: UTC day-start timestamp for the entry. Must align to a day boundary.
- `apy_bps`: `u32`: Gross daily APY. Must be `<= max_apy_bps`.

Accounts

- **oracle_authority**: The APY publisher and rent payer.
 - Signer: Yes.
 - Init: No.
 - PDA: No.

- Writable: Yes (pays rent).
 - Constraints: Must equal `global_config.oracle_authority` (`has_one`).
- **global_config**: Holds the yield index and update cursor.
 - Signer: No.
 - Init: No.
 - PDA: Yes (singleton ["GLOBAL_CONFIG"]; seeds not re-derived here).
 - Writable: Yes.
 - Constraints: `has_one = oracle_authority`.
- **apy_entry**: The per-day APY entry.
 - Signer: No.
 - Init: Yes (`init_if_needed`; a fresh account is a new day, an existing one is a correction).
 - PDA: Yes (seeds ["APY_ENTRY", `day_start_ts.to_le_bytes()`]).
 - Writable: Yes.
 - Constraints: `space = 8 + ApyEntry::INIT_SPACE`; `oracle_authority` pays rent.
- **system_program**: System program.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: Standard program.

Additional checks and behavior

- `get_day_start_ts(day_start_ts) == day_start_ts, else InvalidDayStartTs.`
- `apy_bps <= max_apy_bps, else ApyExceedsMax.` Net APY is `apy_bps` minus the management fee, floored at 0.
- New entry (`global_yield_index_before == 0`): requires `day_start_ts == last_yield_update_ts` (`OutOfOrderApyEntry`) and a full day elapsed (`DayNotElapsed`). Stores the entry, sets `latest_apy_entry`, compounds the global index by one day, and advances `last_yield_update_ts` by one day.
- Correction (existing entry): rewrites that day's net APY, then replays each later entry from `remaining_accounts` in order. Requires the last replayed entry to equal `latest_apy_entry` (`InvalidLatestApyEntry`) and writes the recomputed global index.
- Emits `PostDailyRateEvent`.

CPI

- System program creates the APY entry on a new day.

Instruction: `manager_deposit`

Moves asset tokens from the manager into the asset vault. The manager signs. Emits the manager-deposit event.

Input parameters

- `amount`: u64: Asset amount to deposit. Must be > 0.

Accounts

- **`manager`**: The depositor.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: Must equal `global_config.manager` (has_one); authority of `manager_asset_token_account`.
- **`global_config`**: Backs the manager check and pause state.
 - Signer: No.
 - Init: No.
 - PDA: Yes (singleton ["GLOBAL_CONFIG"]; seeds not re-derived here).
 - Writable: No.
 - Constraints: `has_one = manager`.
- **`asset_config`**: Config for the asset, checked for pause.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds ["ASSET_CONFIG", `asset_mint`], bump = `asset_config.bump`).
 - Writable: No.
 - Constraints: None beyond seeds.
- **`vault_authority`**: Owns the vault ATA.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds ["VAULT_AUTHORITY"]).
 - Writable: No.
 - Constraints: `UncheckedAccount`; address checked by seeds.

- **asset_mint:** The asset mint.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: `address == asset_config.asset_mint, mint::token_program = token_program`.
- **manager_asset_token_account:** The manager's asset source.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: Yes.
 - Constraints: `token::mint = asset_mint, token::authority = manager, matching token program`.
- **asset_vault:** The vault ATA destination.
 - Signer: No.
 - Init: No.
 - PDA: Yes (ATA of `asset_mint / vault_authority`).
 - Writable: Yes.
 - Constraints: `associated_token::authority = vault_authority, matching mint and token program`.
- **token_program:** Token program for the asset mint.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: `Interface<TokenInterface>`.
- **associated_token_program:** Associated Token program.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: Standard program.

Additional checks and behavior

- Rejects when `global_config.paused` (ProgramPaused) or `asset_config.paused` (AssetPaused). Requires `amount > 0`.

- Transfers amount from the manager to the vault. Emits ManagerDepositEvent.
- remaining_accounts: transfer-hook accounts for the transfer.

CPI

- SPL transfer_checked moves the asset manager → vault.

Instruction: manager_withdraw

Moves asset tokens from the vault to the manager when the vault holds enough. The manager signs. Emits the manager-withdraw event.

Input parameters

- amount: u64: Asset amount to withdraw. Must be > 0.

Accounts

- **manager**: The recipient.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Writable: Yes.
 - Constraints: Must equal global_config.manager (has_one); authority of manager_asset_token_account.
- **global_config**: Backs the manager check and pause state.
 - Signer: No.
 - Init: No.
 - PDA: Yes (singleton ["GLOBAL_CONFIG"]; seeds not re-derived here).
 - Writable: No.
 - Constraints: has_one = manager.
- **asset_config**: Config for the asset.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds ["ASSET_CONFIG", asset_mint], bump = asset_config.bump).
 - Writable: No.

- Constraints: None beyond seeds.
- **vault_authority**: Signs the transfer out.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds ["VAULT_AUTHORITY"]).
 - Writable: No.
 - Constraints: UncheckedAccount; address checked by seeds.
- **asset_mint**: The asset mint.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: address == asset_config.asset_mint, mint::token_program = token_program.
- **asset_vault**: The vault ATA source.
 - Signer: No.
 - Init: No.
 - PDA: Yes (ATA of asset_mint / vault_authority).
 - Writable: Yes.
 - Constraints: associated_token::authority = vault_authority, matching mint and token program.
- **manager_asset_token_account**: The manager's asset destination.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: Yes.
 - Constraints: token::mint = asset_mint, token::authority = manager, matching token program.
- **token_program**: Token program for the asset mint.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: Interface<TokenInterface>.
- **associated_token_program**: Associated Token program.
 - Signer: No.
 - Init: No.
 - PDA: No.

- Writable: No.
- Constraints: Standard program.

Additional checks and behavior

- Rejects when `global_config.paused` (`ProgramPaused`). Requires `amount > 0` and `asset_vault.amount >= amount` (`InsufficientVaultBalance`).
- Transfers amount from the vault to the manager, signed by `vault_authority`. Emits `ManagerWithdrawEvent`.
- `remaining_accounts`: transfer-hook accounts for the transfer.

CPI

- SPL `transfer_checked` moves the asset vault → manager, signed by `vault_authority`.

Instruction: `accrue_interest`

Settles a position's matured yield and pending deposits without moving tokens. Anyone calls it. Emits the accrue event.

Input parameters

- None.

Accounts

- `global_config`: Holds the pause state and yield index.
 - Signer: No.
 - Init: No.
 - PDA: Yes (singleton ["GLOBAL_CONFIG"]; seeds not re-derived here).
 - Writable: No.
 - Constraints: None beyond type.
- `treasury_mint`: The receipt mint, read for its decimals.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.

- Constraints: `address == global_config.treasury_mint`.
- **user_position**: The position to settle.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: Yes.
 - Constraints: `AccountLoader<UserPosition>`. No seed or owner check; anyone can settle any position.

Additional checks and behavior

- Rejects when `global_config.paused` (`ProgramPaused`).
- Settles principal, accumulated yield, and matured pending deposits against the posted index, without partial-day extrapolation and without a fallback rate. A missing required APY entry reverts the whole call.
- Writes the updated position. Emits `AccrueInterestEvent` and, when merges occur, `PendingDepositsMergedEvent`.
- `remaining_accounts`: the APY entries needed to merge matured pending deposits.

CPI

- None.

Instruction: `close_user_position`

Closes an empty position back to the user. The user signs. Emits the close event.

Input parameters

- None.

Accounts

- **user**: The position owner and rent recipient.
 - Signer: Yes.
 - Init: No.
 - PDA: No.

- Writable: Yes.
- Constraints: Authority the position is closed to.
- **user_position**: The position being closed.
 - Signer: No.
 - Init: No (closed).
 - PDA: Yes (seeds ["USER_POSITION", user]).
 - Writable: Yes.
 - Constraints: `user_position.owner == user; close = user`.

Additional checks and behavior

- Requires `principal_amount == 0`, `accumulated_yield == 0`, and `pending_deposit_count == 0` (`UserPositionNotEmpty`).
- Emits `CloseUserPositionEvent`.

CPI

- None. Anchor `close` returns the rent to the user.

Instruction: `close_asset_config`

Sweeps any remaining vault balance to the manager, closes the vault and asset config, and decrements `supported_asset_count`. The manager signs. Emits the close event.

Input parameters

- None.

Accounts

- **manager**: The sweep recipient and rent recipient.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Writable: Yes.
 - Constraints: Must equal `global_config.manager` (`has_one`); authority of `manager_asset_token_account`.

- **global_config:** Backs the manager check and holds the counter.
 - Signer: No.
 - Init: No.
 - PDA: Yes (singleton ["GLOBAL_CONFIG"]; seeds not re-derived here).
 - Writable: Yes.
 - Constraints: has_one = manager.
- **asset_config:** The asset config being closed.
 - Signer: No.
 - Init: No (closed).
 - PDA: Yes (seeds ["ASSET_CONFIG", asset_mint], bump = asset_config.bump).
 - Writable: Yes.
 - Constraints: close = manager.
- **vault_authority:** Signs the sweep and vault close.
 - Signer: No.
 - Init: No.
 - PDA: Yes (seeds ["VAULT_AUTHORITY"]).
 - Writable: No.
 - Constraints: UncheckedAccount; address checked by seeds.
- **asset_mint:** The asset mint.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: address == asset_config.asset_mint, mint::token_program = token_program.
- **asset_vault:** The vault ATA, swept and closed.
 - Signer: No.
 - Init: No (closed).
 - PDA: Yes (ATA of asset_mint / vault_authority).
 - Writable: Yes.
 - Constraints: associated_token::authority = vault_authority, matching mint and token program.
- **manager_asset_token_account:** The manager's destination for the swept balance.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: Yes.

- Constraints: token::mint = asset_mint, token::authority = manager, matching token program. Must pre-exist for Token-2022 hooks.
- **token_program**: Token program for the asset mint.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: Interface<TokenInterface>.
- **associated_token_program**: Associated Token program.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: Standard program.
- **system_program**: System program.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: Standard program.

Additional checks and behavior

- Transfers the full vault balance to the manager when it is > 0, signed by vault_authority. Closes the vault ATA. Decrements supported_asset_count (checked subtraction). Closes the asset config to the manager. Emits CloseAssetConfigEvent.
- remaining_accounts: transfer-hook accounts for the sweep.

CPI

- SPL transfer_checked sweeps the vault to the manager. Token close_account closes the vault ATA. Anchor close closes the asset config.

Instruction: close_global_config

Closes the global config once no assets remain and the treasury supply is zero. The manager signs. Emits the close event.

Input parameters

- None.

Accounts

- **manager**: The rent recipient.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Writable: Yes.
 - Constraints: Must equal `global_config.manager` (`has_one`).
- **global_config**: The config being closed.
 - Signer: No.
 - Init: No (closed).
 - PDA: Yes (seeds `["GLOBAL_CONFIG"]`, bump = `global_config.bump`).
 - Writable: Yes.
 - Constraints: `has_one = manager`; `close = manager`.
- **treasury_mint**: The receipt mint, read for its supply.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: `address == global_config.treasury_mint`, `mint::token_program = token_program`.
- **token_program**: Token program for the treasury mint.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Constraints: `Interface<TokenInterface>`.

Additional checks and behavior

- Requires `supported_asset_count == 0` (`SupportedAssetsRemain`) and `treasury_mint.supply == 0` (`TreasuryMintSupplyNotZero`).
- Emits `CloseGlobalConfigEvent`.

CPI

- None. Anchor `close` returns the rent to the manager.
-

Instruction: `prune_apy_entry`

Closes an APY entry after its retention window ends. The manager signs. Emits the prune event.

Input parameters

- `day_start_ts`: `u64`: Day-start timestamp identifying the APY entry, used as the seed.

Accounts

- **`manager`**: The rent recipient.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Writable: Yes.
 - Constraints: Must equal `global_config.manager` (`has_one`).
- **`global_config`**: Holds the retention duration and the latest-entry reference.
 - Signer: No.
 - Init: No.
 - PDA: Yes (singleton ["GLOBAL_CONFIG"]; seeds not re-derived here).
 - Writable: Yes.
 - Constraints: `has_one` = `manager`.
- **`apy_entry`**: The entry being closed.
 - Signer: No.
 - Init: No (closed).
 - PDA: Yes (seeds ["APY_ENTRY", `day_start_ts.to_le_bytes()`], `bump` = `apy_entry.bump`).
 - Writable: Yes.
 - Constraints: `close` = `manager`.

Additional checks and behavior

- Requires `apy_entry.key() != global_config.latest_apy_entry`

(CannotPruneLatestApyEntry).

- Requires `now >= apy_entry.day_start_ts + apy_history_retention_duration` (ApyEntryRetentionNotMet).
- Closing an entry still referenced by an unsettled pending deposit blocks that position from settling, because the merge needs the entry.
- Emits `PruneApyEntryEvent`.

CPI

- None. Anchor `close` returns the rent to the manager.

5. Assessment Results

During our assessment on the scoped Multiliquid SVM programs, we discovered 19 findings. No critical issues were found. Four findings were of medium impact, three were of low impact, and the remaining findings were informational in nature.

5.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Where Zellic states that a finding has been addressed or fixed in one or more specific commits, this indicates only that those commits introduce changes that, in our assessment, address the finding relative to the codebase version originally reviewed. This does not imply that Zellic reviewed other changes contained in those commits, the overall state of the codebase at those commits, or the interplay between remediation measures for different findings.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.